



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN TECNOLOXÍAS DA INFORMACIÓN



Diseño y Desarrollo de un Datamart Analítico para la Prevención del Abandono y Fracaso Escolar

Estudiante: Andrés Paz Paredes
Dirección: José Ramón Paramá Gabía

A Coruña, junio de 2026.

Dedicatoria

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Resumen

Este Trabajo de Fin de Grado presenta el diseño y desarrollo de un Datamart Analítico para la prevención del abandono escolar. El sistema integra datos académicos y sociodemográficos desde sistemas transaccionales, utilizando Apache Hop (ETL) para procesarlos hacia un Data Warehouse en PostgreSQL. Todo este ecosistema tecnológico se ha orquestado mediante contenedores Docker, garantizando su portabilidad y escalabilidad. Durante el proceso, se calculan métricas de riesgo predictivo para identificar alumnos vulnerables. La solución final expone esta información mediante una API en Python (FastAPI) y la visualiza en un Dashboard interactivo en React, proporcionando a los centros educativos una herramienta eficaz para la toma de decisiones basada en datos.

Abstract

This Bachelor's Thesis presents the design and development of an Analytical Datamart for the prevention of school dropout. The system integrates academic and socio-demographic data from transactional systems, using Apache Hop (ETL) to process them into a Data Warehouse on PostgreSQL. This entire technological ecosystem has been orchestrated using Docker containers, ensuring its portability and scalability. During the process, predictive risk metrics are calculated to identify vulnerable students. The final solution exposes this information through a Python API (FastAPI) and visualizes it in an interactive React Dashboard, providing educational centers with an effective tool for data-driven decision-making.

Palabras clave:

- Inteligencia de Negocios
- Prevención Abandono Escolar
- Datamart
- ETL
- Apache Hop
- Contenedores Docker

Keywords:

- Business Intelligence
- School Dropout Prevention
- Datamart
- ETL
- Apache Hop
- Docker Containers

Índice general

1	Introducción	1
1.1	Contexto	1
1.2	Motivación	2
1.3	Objetivos	2
2	Estado del arte y marco teórico	4
2.1	Inteligencia de negocios y data warehousing	4
2.1.1	Metodología de Ralph Kimball	4
2.1.2	Visualización de datos y cuadros de mando	6
2.2	Procesos ETL y Apache Hop	6
2.2.1	Elección de Apache Hop	6
2.3	Arquitecturas web: cliente-servidor y APIs REST	6
2.4	Evolución de las arquitecturas: contenedores	7
3	Metodología y planificación	8
3.1	Metodología de desarrollo	8
3.2	Gestión y control de versiones	9
3.3	Planificación temporal	9
4	Análisis y arquitectura	10
4.1	Requisitos del sistema	10
4.2	Análisis del sistema transaccional origen	11
4.3	Arquitectura tecnológica basada en Docker	11
4.3.1	Justificación del paradigma de contenedores	14
5	Diseño del datamart analítico	16
5.1	Modelo dimensional en estrella	16
5.1.1	Tablas de dimensiones en detalle	16

5.1.2	Tablas de hechos	17
5.2	Reglas de negocio y cálculo de riesgos	17
5.2.1	Riesgo socioeconómico	17
5.2.2	Riesgo de abandono del alumno	18
6	Capa ETL con Apache Hop	20
6.1	Configuración del entorno y conexiones	20
6.2	Análisis de pipelines	20
6.2.1	Volcado a staging	21
6.2.2	Dimensión de demografía familiar	21
6.2.3	Dimensión de estudiante	23
6.2.4	Dimensión de tiempo	25
6.2.5	Dimensión de asignatura	26
6.2.6	Dimensión de curso	27
6.2.7	Dimensión de adaptación	29
6.2.8	Dimensión de centro	30
6.2.9	Tabla de hechos de calificaciones	31
6.2.10	Tabla de hechos de rendimiento por evaluación	32
6.3	Orquestación y workflows	36
6.3.1	Propagación de parámetros dinámicos	37
6.3.2	Manejo de errores multinivel (<i>error handling</i>)	41
7	Dashboard analítico	44
7.1	Servidor backend RESTful (FastAPI)	44
7.1.1	Mapeo objeto-relacional (SQLAlchemy)	44
7.1.2	Estructura de endpoints y seguridad	45
7.2	Interfaz visual e interacción (React y Vite)	45
7.2.1	Sistema de diseño: <i>Glassmorphism</i> y temas dinámicos	45
7.2.2	Visualización de datos analíticos	46
7.3	Despliegue y enrutamiento web (Nginx)	46
7.3.1	Arquitectura de interceptación de tráfico	46
8	Conclusiones	47
A	Material adicional	50
	Bibliografía	52

Índice de figuras

2.1	Esquema conceptual de un modelo en estrella.	5
2.2	Interfaz de usuario de Apache Hop.	7
3.1	Diagrama de Gantt de la planificación del proyecto.	9
4.1	Modelo Entidad-Relación (ER) de la base de datos transaccional origen.	12
4.2	Esquema Relacional de origen.	13
6.1	Configuración de la conexión <i>postgres-dwh</i>	21
6.2	Flujo del pipeline <i>extract_all_to_staging.hpl</i>	22
6.3	Flujo del pipeline <i>dim_demografia_familiar.hpl</i>	24
6.4	Flujo del pipeline <i>dim_estudiante.hpl</i>	25
6.5	Flujo del pipeline <i>dim_tiempo.hpl</i>	27
6.6	Flujo del pipeline <i>dim_asignatura.hpl</i>	28
6.7	Flujo del pipeline <i>dim_curso.hpl</i>	29
6.8	Flujo del pipeline <i>dim_adaptacion.hpl</i>	30
6.9	Flujo del pipeline <i>dim_centro.hpl</i>	31
6.10	Flujo del pipeline <i>fact_calificaciones.hpl</i> (Parte 1).	33
6.11	Flujo del pipeline <i>fact_calificaciones.hpl</i> (Parte 2).	34
6.12	Flujo del pipeline <i>fact_rendimiento_anual_full.hpl</i> (Parte 1).	37
6.13	Flujo del pipeline <i>fact_rendimiento_anual_full.hpl</i> (Parte 2).	38
6.14	Flujo completo del pipeline <i>fact_rendimiento_anual_incremental.hpl</i>	39
6.15	Nodo <i>Database Join</i> en <i>fact_rendimiento_anual_incremental.hpl</i>	40
6.16	Workflow principal (<i>main.hwf</i>) y su delegación jerárquica en sub-flujos.	42
6.17	Sub-flujo de dimensiones (<i>dimensions.hwf</i>) ejecutado en paralelo.	42
6.18	Sub-flujo de hechos (<i>facts.hwf</i>) ejecutado de forma secuencial.	43

Índice de tablas

Introducción

EL abandono temprano de la educación tiene importantes implicaciones sociales en términos de calidad de vida, salud, marginalidad social, delincuencia o empleabilidad. Dado el importante coste socioeconómico que este fenómeno conlleva a largo plazo, todas las administraciones públicas y organismos gubernamentales tratan de reducir en lo posible la tasa de abandono temprano y fracaso escolar mediante políticas de intervención y apoyo proactivo [1].

1.1 Contexto

La administración pública, habitualmente a nivel de comunidad autónoma, es la entidad encargada de la gestión de la educación y debe enfrentarse al complejo desafío de realizar un reparto equitativo de los escasos recursos materiales y personales entre su extensa red de centros educativos.

La casuística de cada institución es única, y algunos centros necesitan un volumen de apoyo significativamente mayor que otros debido a múltiples factores sistémicos y estructurales. Estar ubicados en zonas económicamente deprimidas, contar con un alto porcentaje de alumnado proveniente de clases desfavorecidas o enfrentarse a severas barreras de brecha digital son elementos que lastran el rendimiento global de un centro. Esto se alinea con la realidad del indicador AROPE en España, que evidencia cómo un porcentaje significativo de las familias conviven en situaciones de vulnerabilidad o pobreza relativa [2, 3]. Abordar estas carencias desde el prisma de la "pobreza multidimensional" [4, 5] es fundamental para que la administración actúe como un equilibrador social efectivo.

Sin embargo, históricamente la información ha estado dispersa en bases de datos heterogéneas a nivel gubernamental. Para hacer un reparto verdaderamente objetivo y eficiente, la administración no puede depender de intuiciones o reportes fragmentados; debe disponer de una "fotografía" exacta del estado de cada centro, construida sobre medidas estandarizadas,

comparables y objetivas a nivel autonómico.

1.2 Motivación

Ante este escenario de gran volumen y dispersión de datos, surge la necesidad imperativa de aplicar técnicas de Inteligencia de Negocios (*Business Intelligence* o BI) a la gestión pública. Dado el inmenso número de centros educativos que suelen estar bajo la tutela de las administraciones, la toma de decisiones manual y aislada resulta inabarcable.

La motivación fundamental de este Trabajo de Fin de Grado es el diseño y desarrollo de un ecosistema analítico que culmine en la creación de un cuadro de mando. Dicho cuadro de mando estará compuesto por una serie de Indicadores Clave de Desempeño (KPIs, por sus siglas en inglés) que sinteticen y crucen de forma inteligente el rendimiento académico con el contexto sociodemográfico de los centros a escala global.

El propósito final es que la tecnología se ponga al servicio de los gestores públicos, sirviendo como una herramienta centralizada y accionable para guiar la difícil tarea de asignación de recursos, distribución de presupuestos y dotación de personal de apoyo entre los distintos centros educativos de forma totalmente empírica y justificada.

1.3 Objetivos

El objetivo principal de este proyecto es crear un almacén de datos (Data Warehouse) que extraiga su información a partir de diversas fuentes institucionales, y el desarrollo de un cuadro de mando que muestre distintos KPIs de interés estratégico para los gestores públicos.

Para alcanzar este propósito, garantizando además que todo el sistema se diseñe utilizando exclusivamente herramientas libres de licencias de pago, el trabajo se ha desglosado en los siguientes objetivos específicos:

1. **Simulación de fuentes de datos:** Crear tres bases de datos de origen que representen el entorno autonómico: una con expedientes académicos de alumnos, una con adaptaciones curriculares y una tercera con los resultados de encuestas socioeconómicas familiares.
2. **Diseño del almacén de datos:** Modelar un repositorio centralizado bajo un enfoque dimensional (Datamart) que aglutine e integre la información de las tres fuentes mencionadas para ser el origen analítico de los KPIs.
3. **Diseño de procesos ETL:** Diseñar e implementar los flujos de Extracción, Transformación y Carga (ETL) que pueblen los datos de las fuentes de origen hacia el almacén de datos.

4. **Desarrollo de la lógica de negocio (Backend):** Desplegar una API robusta y segura que sirva como capa de servicio estandarizada entre el almacén de datos y la interfaz gráfica.
5. **Creación de la aplicación web (Dashboard):** Implementar el cuadro de mando interactivo destinado a los gestores, diseñado para mostrar y explorar visualmente los KPIs desglosados por centro, localidad y período de tiempo.

De este modo, el proyecto abarca de manera secuencial e iterativa la totalidad del ciclo de vida de un proyecto de Inteligencia de Negocios aplicado a la administración pública.

Estado del arte y marco teórico

PARA comprender adecuadamente las decisiones arquitectónicas y técnicas tomadas en este proyecto, es fundamental contextualizar los conceptos teóricos subyacentes. Este capítulo describe los fundamentos de la Inteligencia de Negocios y las herramientas ETL que sustentan el Datamart desarrollado.

2.1 Inteligencia de negocios y data warehousing

La Inteligencia de Negocios (*Business Intelligence* o BI) comprende el conjunto de metodologías, aplicaciones y tecnologías que permiten reunir, depurar y transformar datos de sistemas transaccionales en información estructurada, para su explotación directa o para su análisis y conversión en conocimiento.

En el ámbito de la gestión educativa pública, la aplicación de la Inteligencia de Negocios trasciende el mero análisis de expedientes individuales para convertirse en una herramienta estratégica de gobernanza. La explotación masiva de datos a nivel autonómico o gubernamental permite a las administraciones abandonar los modelos de reparto de recursos basados en la inercia institucional o informes fragmentados. Al aplicar técnicas de BI sobre bases de datos demográficas, económicas y académicas consolidadas, los gestores públicos pueden identificar patrones de vulnerabilidad sistémica, comparar objetivamente las necesidades entre distintos centros y diseñar políticas educativas proactivas fundamentadas en evidencias empíricas (*Data-Driven Policy Making*).

2.1.1 Metodología de Ralph Kimball

Para el diseño del repositorio de datos se ha optado por la metodología ascendente o *Bottom-Up* propuesta por Ralph Kimball [6]. A diferencia de enfoques transaccionales clásicos, Kimball propone construir el almacén de datos (Data Warehouse) a partir de la integración de diferentes *Datamarts* temáticos.

Un Datamart es un subconjunto orientado a una área específica (en este caso, la orientación y prevención del abandono escolar). La estructura central de esta metodología es el modelo dimensional, habitualmente implementado mediante esquemas en estrella (ver Figura 2.1). Para comprender este diseño, es esencial entender dos conceptos arquitectónicos clave:

- **Modelo en Estrella:** Consiste en separar radicalmente la información en dos tipos de tablas. En el centro de la "estrella" se sitúan las *Tablas de Hechos* (*Fact Tables*), que almacenan eventos medibles y cuantitativos (por ejemplo, las calificaciones o el riesgo de abandono). Alrededor, conectadas directamente a la tabla de hechos, orbitan las *Tablas de Dimensiones* (*Dimension Tables*), que contienen el contexto descriptivo de esos eventos (el "quién", "cuándo", "dónde" y "qué": estudiante, tiempo, centro, curso).
- **Desnormalización Intencionada:** Mientras que las bases de datos operativas tradicionales (OLTP) aplican la "normalización" (dividir los datos en múltiples tablas para evitar redundancias y proteger la escritura), los Datamarts (OLAP) hacen exactamente lo contrario. Se aplica la *desnormalización*, es decir, se admite la redundancia de datos (aplanando jerarquías como "Ciclo → Especialidad → Curso" en una sola tabla) con el objetivo de reducir drásticamente las operaciones matemáticas de cruce (JOINS). Esto permite que las consultas analíticas sobre grandes volúmenes de datos sean extremadamente rápidas para alimentar cuadros de mando.



Figura 2.1: Esquema conceptual de un modelo en estrella.

2.1.2 Visualización de datos y cuadros de mando

El objetivo final de cualquier arquitectura de Inteligencia de Negocios es la correcta comunicación de los hallazgos a los responsables de la toma de decisiones. Teóricamente, un *Dashboard* o Cuadro de Mando Integral es una interfaz visual que proporciona un resumen centralizado de los Indicadores Clave de Rendimiento (KPIs). El estado del arte en el diseño de estas interfaces aboga por principios de "carga cognitiva reducida", priorizando la legibilidad, el uso de jerarquías de información y el empleo estratégico del color para señalar desviaciones críticas. Además, es fundamental la capacidad de *drill-down* (profundización analítica), que permite al gestor público navegar de forma fluida desde una visión macroscópica a nivel de toda la comunidad o municipio, hasta diseccionar el estado de vulnerabilidad de un centro educativo específico.

2.2 Procesos ETL y Apache Hop

Los procesos ETL (Extracción, Transformación y Carga) son el motor de cualquier sistema BI. Son los encargados de extraer los datos de las fuentes originales, aplicarles reglas de limpieza, filtrado y transformación, y finalmente cargarlos en el Datamart dimensional.

2.2.1 Elección de Apache Hop

Apache Hop (*Hop Orchestration Platform*) [7] es una plataforma moderna de ingeniería de datos y orquestación. Nace como una bifurcación (*fork*) de Pentaho Data Integration (PDI), con el objetivo de modernizar su arquitectura, mejorar su integración con entornos de contenedores y facilitar su uso a través de una interfaz de usuario web (*Hop Web*).

La elección de Apache Hop frente a otras herramientas del mercado se fundamenta en su curva de aprendizaje, su modelo visual *drag-and-drop* (ver Figura 2.2) y, especialmente, su excelente soporte para la ejecución nativa en contenedores Docker, lo cual encaja perfectamente con la arquitectura diseñada para este proyecto.

2.3 Arquitecturas web: cliente-servidor y APIs REST

El diseño de aplicaciones analíticas modernas ha abandonado los modelos donde el servidor generaba directamente las vistas que el usuario consumía. El marco teórico actual se fundamenta en el paradigma **Cliente-Servidor desacoplado**. En este modelo, el *Frontend* (interfaz de usuario) y el *Backend* (lógica de negocio y base de datos) operan como aplicaciones completamente independientes.

Para lograr la comunicación entre ambas capas, el estándar universal son las **APIs RESTful** (*Representational State Transfer*). Una API REST es una interfaz que utiliza el protocolo

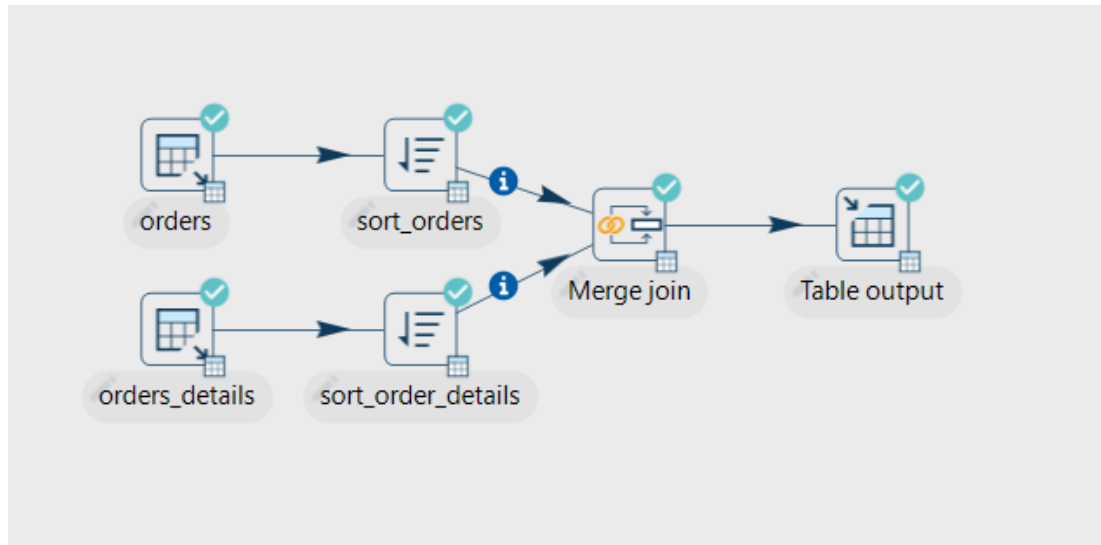


Figura 2.2: Interfaz de usuario de Apache Hop.

HTTP para solicitar y enviar datos. Bajo este enfoque, el servidor expone *endpoints* a los que el cliente realiza peticiones para obtener la información analítica. Las respuestas se estructuran habitualmente en formato JSON (*JavaScript Object Notation*), un estándar ligero que permite al *Frontend* procesar ágilmente los datos y renderizarlos en gráficos interactivos. Este desacoplamiento es esencial para asegurar la escalabilidad del sistema.

2.4 Evolución de las arquitecturas: contenedores

El estado del arte en el despliegue de infraestructuras de datos ha sufrido un cambio de paradigma radical en los últimos años. Tradicionalmente, proyectos complejos que involucraban múltiples bases de datos, flujos ETL y servidores web se desplegaban como arquitecturas monolíticas o empleaban máquinas virtuales que consumían ingentes recursos del sistema operativo anfitrión.

La introducción de la **contenedorización**, abanderada por plataformas como Docker, representa el estándar actual de la industria. Teóricamente, un contenedor encapsula un microservicio individual junto con todas sus librerías y archivos de configuración, aislándolo completamente. Esta filosofía de "aislamiento de procesos ligeros" garantiza la inmutabilidad del software (eliminando el clásico problema de diferencias entre entornos de desarrollo y producción). Para un ecosistema educativo, donde la infraestructura tecnológica de los colegios puede ser muy limitada y heterogénea, apoyarse en este marco teórico es indispensable para asegurar un despliegue reproducible, ágil e independiente del hardware subyacente.

Metodología y planificación

EL desarrollo de un proyecto de software de la envergadura de un Trabajo de Fin de Grado requiere una organización rigurosa para garantizar el cumplimiento de los plazos y la calidad de los entregables.

3.1 Metodología de desarrollo

Debido a la naturaleza investigadora y de desarrollo individual de este proyecto, no se ha adoptado una metodología ágil estricta (como Scrum), que está más orientada a equipos de desarrollo con roles definidos.

En su lugar, se ha optado por un enfoque *iterativo e incremental*. Este enfoque permite dividir el ciclo de vida del proyecto en ciclos más cortos y manejables (iteraciones), en cada uno de los cuales se añade funcionalidad o se refinan las características existentes.

Las fases iterativas de este proyecto han sido las siguientes:

1. **Análisis e investigación teórica:** Definición de los factores de riesgo de abandono escolar y estudio de la viabilidad de extracción de datos.
2. **Diseño de la arquitectura:** Elección de la pila tecnológica y diseño del ecosistema de contenedores Docker.
3. **Modelado de datos:** Diseño del esquema en estrella y mapeo origen-destino.
4. **Desarrollo ETL:** Implementación iterativa de los pipelines (extracción de tablas transaccionales, transformación y carga dimensional).
5. **Desarrollo Backend y Frontend:** Exposición de los datos mediante una API RESTful y creación de los *dashboards* visuales, ajustando los umbrales de alerta en base a los resultados observados.

Esta estrategia ha dotado al desarrollo de la flexibilidad necesaria para ajustar las reglas de negocio, como las puntuaciones de riesgo por necesidades educativas especiales o los umbrales socioeconómicos, a medida que se profundizaba en la casuística de los datos reales.

3.2 Gestión y control de versiones

Para el correcto seguimiento de la evolución del proyecto, se ha empleado **Git** como sistema de control de versiones descentralizado. El repositorio central se ha alojado en GitHub, lo que ha facilitado mantener un histórico organizado de los cambios estructurales.

Siguiendo las mejores prácticas de la industria, el repositorio se estructuró monorepo separando claramente los dominios funcionales en directorios independientes (*api*, *dwh*, *hop*, *frontend*), lo que permitió aplicar la metodología iterativa sobre componentes aislados sin interferir con el resto del ecosistema.

3.3 Planificación temporal

Para el control y la gestión del tiempo del proyecto, se ha empleado un diagrama de Gantt (ver Figura 3.1), donde se detallan las distintas iteraciones de desarrollo a lo largo de los cuatro meses de duración del Trabajo de Fin de Grado.

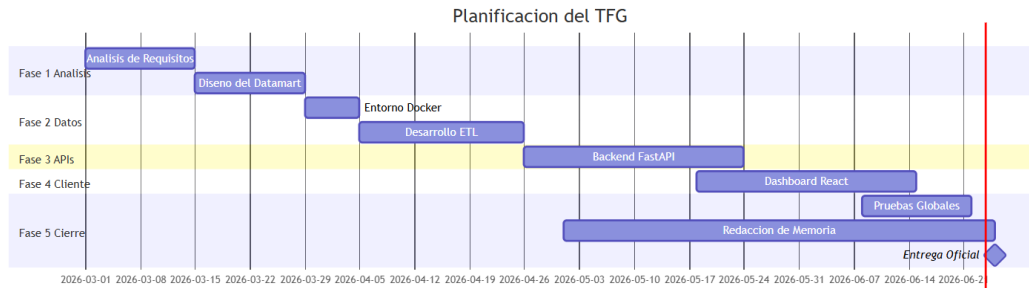


Figura 3.1: Diagrama de Gantt de la planificación del proyecto.

Análisis y arquitectura

EN este capítulo se detallan los requisitos del sistema y la arquitectura tecnológica seleccionada para la implementación del Datamart y el cuadro de mando. Es importante destacar que todo el proyecto se ha diseñado y desarrollado utilizando exclusivamente tecnologías **Open Source** (código abierto). Esta decisión estratégica garantiza que cualquier centro educativo, sea público o privado, pueda adoptar y desplegar esta solución sin incurrir en costes de licenciamiento de software comercial.

4.1 Requisitos del sistema

Para lograr el objetivo de detectar de forma temprana el riesgo de abandono escolar y la brecha digital, el sistema debe cumplir con los siguientes requisitos funcionales y no funcionales:

- **RF1 - Extracción y Transformación:** El sistema debe ser capaz de extraer datos en bruto del sistema transaccional (PostgreSQL), limpiarlos y aplicar reglas de negocio para calcular índices de riesgo socioeconómico y académico.
- **RF2 - Modelo Dimensional:** Los datos deben estructurarse en un Data Warehouse bajo un modelo de estrella para facilitar consultas analíticas ultrarrápidas y cruces demográficos.
- **RF3 - Visualización y Alertas:** Se debe proveer una interfaz web (Dashboard) que muestre de forma intuitiva qué centros educativos o cohortes geográficas presentan un riesgo alto de vulnerabilidad sistémica y fracaso escolar.
- **RNF1 - Modularidad y Escalabilidad:** La arquitectura debe estar dividida en microservicios independientes, facilitando su despliegue y mantenimiento.

- **RNF2 - Tecnologías Libres:** Obligatoriedad de utilizar herramientas Open Source en todas las capas del proyecto.

4.2 Análisis del sistema transaccional origen

Para comprender la necesidad de construir un Data Warehouse y diseñar los flujos ETL, es fundamental analizar previamente la estructura de la base de datos origen. En este proyecto se ha simulado una base de datos transaccional (OLTP) en PostgreSQL que replica el sistema de gestión académica y administrativa estándar de los centros educativos.

Este sistema está altamente normalizado para garantizar la integridad referencial y evitar la redundancia de datos en las operaciones diarias (altas de matrículas, registro de calificaciones, encuestas familiares). El modelo relacional consta de 14 tablas principales interconectadas (ver Figura 4.1 y Figura 4.2), agrupadas en los siguientes dominios lógicos:

- **Dominio Académico:** Tablas *CICLO_EDUCATIVO*, *ESPECIALIDAD*, *CURSO* y *ASIGNATURA*. Definen el catálogo formativo del sistema.
- **Dominio de Matriculación y Evaluación:** Tablas *CENTRO*, *ADMISION*, *MATRICULA*, *ASIGNATURA_MATRICULA* y la entidad débil *LINEA_EXPEDIENTE*, que almacena el histórico de calificaciones por evaluación.
- **Dominio del Alumnado y Diversidad:** Tablas *ESTUDIANTE*, *ADAPTACION_CURRICULAR* y *ADAPTACION_ASIGNATURA*, registrando los expedientes y las medidas de apoyo educativo.
- **Dominio Socioeconómico:** Tablas *RESPONSABLE_LEGAL* y *ENCUESTA*. Contienen el histórico de ingresos, acceso a internet y nivel educativo de los progenitores; datos vitales para calcular posteriormente el riesgo socioeconómico.

Dada la alta fragmentación de la información (por ejemplo, para relacionar las calificaciones de un estudiante con el nivel de renta de sus tutores es necesario cruzar simultáneamente las tablas *LINEA_EXPEDIENTE*, *ASIGNATURA_MATRICULA*, *MATRICULA*, *ESTUDIANTE*, *RESPONSABLE_LEGAL* y *ENCUESTA*), ejecutar algoritmos analíticos directamente sobre este esquema penalizaría severamente el rendimiento del sistema en producción. Esta limitación inherente a los sistemas OLTP justifica plenamente el diseño dimensional del Datamart.

4.3 Arquitectura tecnológica basada en Docker

Para cumplir con el requisito de modularidad, la arquitectura del proyecto se ha construido bajo el paradigma de contenedores utilizando **Docker** [8]. Esto permite aislar cada com-

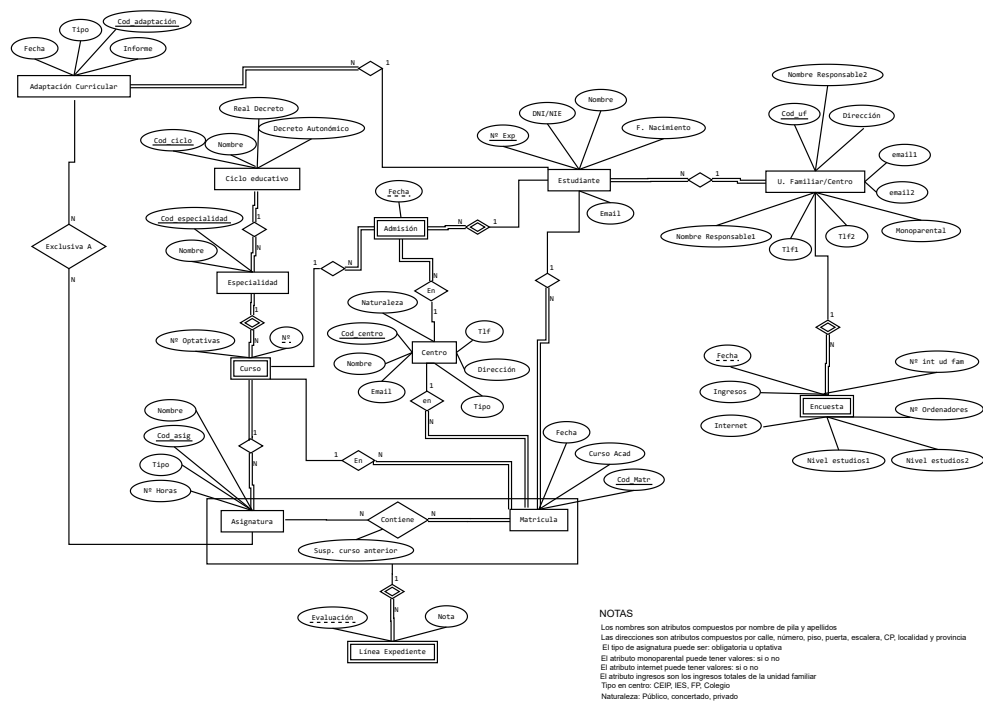


Figura 4.1: Modelo Entidad-Relación (ER) de la base de datos transaccional origen.

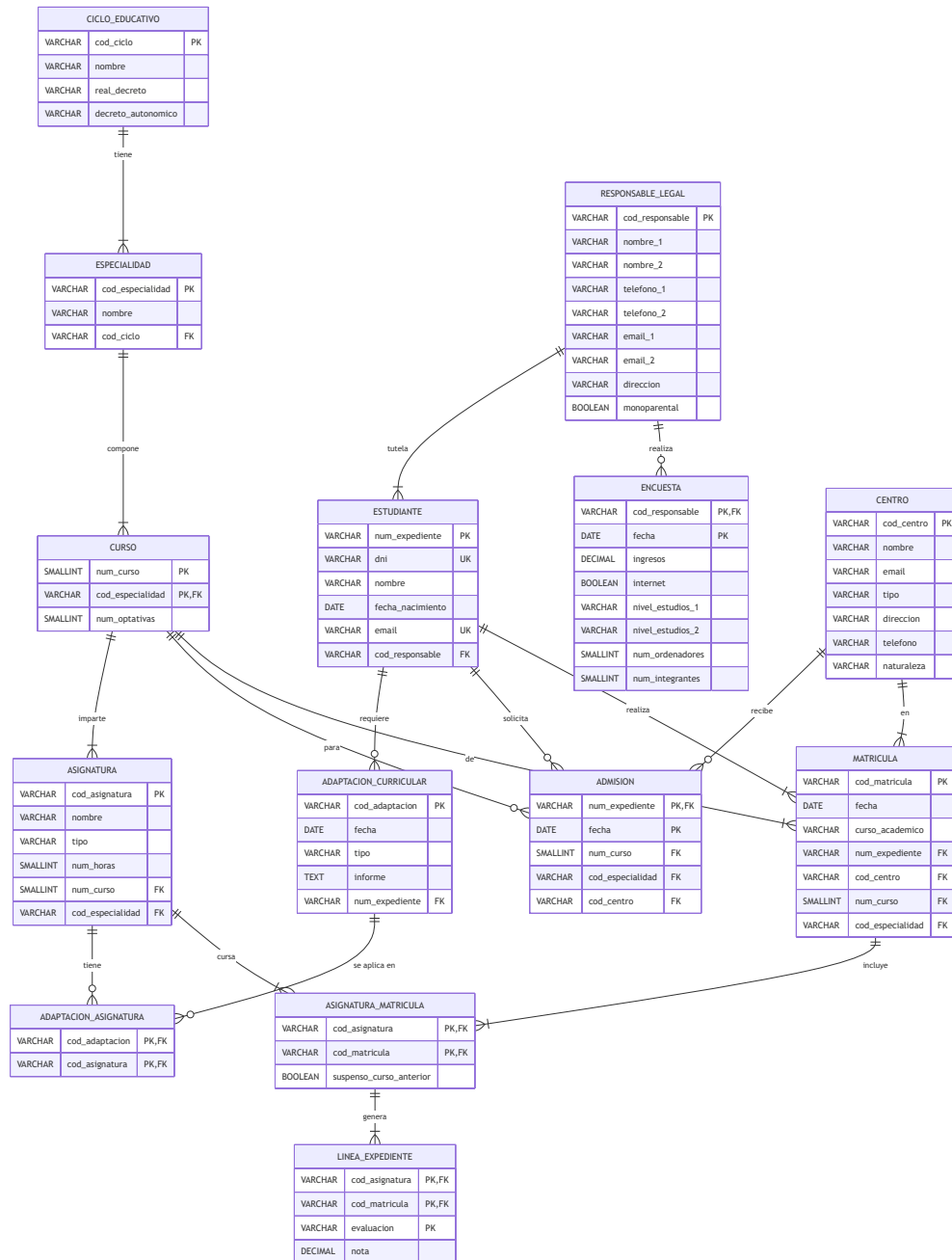


Figura 4.2: Esquema Relacional de origen.

ponente del sistema en su propio entorno, asegurando que funcionen de manera predecible independientemente del servidor donde se desplieguen.

El ecosistema está compuesto por seis contenedores principales orquestados mediante *docker-compose*:

1. **Origen de Datos Transaccional (PostgreSQL) [9]**: Simula la base de datos operativa (*source*) de los centros educativos. Contiene tablas normalizadas con los expedientes, matrículas, notas y encuestas familiares en tiempo real.
2. **Data Warehouse Analítico (PostgreSQL)**: Actúa como el repositorio central de BI (*dwh*). Almacena los datos estructurados en un modelo dimensional (esquema en estrella) optimizado para lectura y agregación, conteniendo el historial acumulado.
3. **Orquestador ETL (Apache Hop Web)**: Contenedor que ejecuta los flujos de transformación (*pipelines*). Se encarga de conectarse al Origen de Datos, extraer la información, calcular las métricas de riesgo (socioeconómico y de abandono) y cargar el resultado en el Data Warehouse.
4. **Servidor Backend RESTful (FastAPI) [10]**: Construido en Python, este contenedor expone una API segura que se conecta al Data Warehouse, realiza las consultas analíticas pesadas mediante *SQLAlchemy* y sirve los resultados en formato JSON al cliente.
5. **Aplicación Cliente Dashboard (React/Vite) [11]**: Interfaz gráfica web moderna donde los gestores de la administración visualizan los datos. Utiliza *Recharts* para los gráficos y sigue una estética visual basada en *Glassmorphism*.
6. **Servidor Web y Proxy Inverso (Nginx)**: Se encarga de servir la aplicación React compilada y de enrutar las peticiones de red hacia la API del Backend, asegurando un único punto de entrada seguro para la aplicación.

Esta separación clara entre la base de datos transaccional (OLTP) y el almacén analítico (OLAP) garantiza que las complejas consultas del Dashboard no penalicen el rendimiento de las operaciones diarias del colegio.

4.3.1 Justificación del paradigma de contenedores

Aunque el diseño de una arquitectura distribuida conlleva una mayor carga de razonamiento inicial respecto a una solución monolítica, esta decisión aporta un enorme valor añadido al proyecto por tres motivos fundamentales:

- **Aislamiento y Reproducibilidad**: Cada contenedor empaqueta sus propias dependencias (por ejemplo, librerías concretas de Python para FastAPI o módulos de Node

para React). Esto elimina el clásico problema de *"funciona en mi máquina"* y garantiza que el comportamiento del software sea idéntico en desarrollo, pruebas y producción.

- **Despliegue Universal (*One-Click Deployment*):** Al estar orientado a centros educativos (instituciones que frecuentemente carecen de departamentos de IT especializados o presupuestos para infraestructuras complejas), la orquestación con *docker-compose* permite levantar toda la plataforma (bases de datos, ETL, API y web) con un único comando. Esto facilita enormemente su transferencia tecnológica y adopción real.
- **Topología de Red y Seguridad:** Docker permite crear redes virtuales cerradas (*Docker Networks*). De esta forma, bases de datos críticas como el Data Warehouse y los flujos de transformación de Apache Hop quedan totalmente invisibles desde el exterior. Solamente el proxy inverso (Nginx) expone los puertos estrictamente necesarios, añadiendo una capa de seguridad pasiva esencial cuando se tratan datos académicos y demográficos de menores.

Diseño del datamart analítico

EL diseño del Datamart constituye el núcleo lógico de este proyecto. Se ha optado por un modelo dimensional en estrella (Star Schema) basado en la metodología de Ralph Kimball, pero introduciendo optimizaciones específicas para tratar variables demográficas de alta cardinalidad e historificación de expedientes.

5.1 Modelo dimensional en estrella

A diferencia de las bases de datos transaccionales, que están altamente normalizadas, el esquema del Data Warehouse se ha desnormalizado en dos tipos de tablas: **Tablas de Hechos** (que almacenan métricas y claves foráneas) y **Tablas de Dimensiones** (que contienen descriptores textuales para filtrar y agrupar).

En nuestro diseño final se han establecido dos tablas de hechos de diferente granularidad para responder a distintas necesidades analíticas, rodeadas por un total de siete dimensiones.

5.1.1 Tablas de dimensiones en detalle

Las dimensiones contextualizan los datos numéricos. Para este proyecto se han diseñado las siguientes:

- ***dim_estudiante* (SCD Tipo 2):** Contiene los datos personales del alumno y su responsable legal. Para reflejar la inestabilidad habitacional o cambios de tutela (muy comunes en entornos vulnerables), se implementó una *Dimensión de Variación Lenta de Tipo 2* (SCD2). Esto significa que ante un cambio de tutor o domicilio, no se sobrescribe la fila, sino que se caduca (marcando una fecha de fin) y se crea una nueva, preservando el contexto histórico de las notas de años anteriores.
- ***dim_centro*:** Almacena la información institucional y geográfica de los colegios e institutos.

- ***dim_curso***: Consolida la estructura académica (Ciclo, Especialidad y Curso).
- ***dim_asignatura***: Catálogo de materias independientes del currículo, con su carga horaria.
- ***dim_tiempo***: Dimensión temporal que almacena el año académico y el periodo de evaluación.
- ***dim_demografia_familiar***: Utiliza un modelo demográfico híbrido para reducir la cardinalidad. En lugar de almacenar ingresos exactos, clasifica a las familias por su nivel de renta en tramos, conectividad a internet y disponibilidad de ordenadores.
- ***dim_adaptacion***: Registra si el alumno cuenta con adaptaciones curriculares, categorizándolas por discapacidad médica o compensación sociopedagógica.

5.1.2 Tablas de hechos

El modelo implementa dos niveles de análisis a través de dos tablas de hechos:

- ***fact_calificaciones* (Granularidad Atómica)**: Representa el nivel más detallado. Cada fila es la nota exacta de un estudiante en una asignatura durante una evaluación concreta. Es útil para rastrear el progreso pormenorizado a lo largo del curso.
- ***fact_rendimiento_anual* (Granularidad Agregada)**: Cada fila representa el resumen académico total de un alumno en un curso completo. Consolida el número de asignaturas matriculadas, aprobadas, la tasa de éxito, las asignaturas arrastradas de años anteriores y si el alumno ha repetido. Es en esta tabla donde se calculan de manera definitiva los KPIs críticos y el índice de Riesgo de Abandono Escolar Temprano.

5.2 Reglas de negocio y cálculo de riesgos

Para que el Dashboard ofrezca métricas accionables a nivel autonómico, se han parametrizado dos indicadores de riesgo fundamentales mediante sistemas de puntuación acumulativa, integrados en la lógica del Datamart.

5.2.1 Riesgo socioeconómico

Este riesgo evalúa el contexto familiar de exclusión y brecha digital. Los puntos se calculan sumando los siguientes factores:

- **Estudios de los Progenitores (Estándar PARED)**: Se ha adoptado el enfoque de la OCDE en sus informes PISA, donde el riesgo correlaciona con el mayor nivel educativo

alcanzado por cualquiera de los padres [12]. Puntuaciones aplicadas a cada progenitor: Sin estudios de primaria (5 pts), Primaria (3 pts), Secundaria o superior (1 pt).

- **Renta por Unidades de Consumo (Escala OCDE Modificada):** Para determinar el riesgo económico no se divide la renta de forma puramente lineal (per cápita). Siguiendo la metodología AROPE de Eurostat y las directrices del INE [13, 14], la escala teórica estipula que el primer adulto computa como 1 unidad de consumo, los integrantes mayores de 14 años como 0,5 unidades y los menores de 14 años como 0,3. No obstante, debido a que el sistema transaccional de origen no recopila la edad exacta del resto de convivientes (hermanos o familiares), ha sido necesario aplicar una adaptación metodológica en la ETL. Se asigna un peso de 1 al primer integrante y de 0,5 al resto, independientemente de su edad. En base a los ingresos anuales equivalentes obtenidos tras la ponderación, se establecen los siguientes niveles de renta en la capa de transformación:
 - **Muy Baja (Pobreza severa):** Ingresos por unidad de consumo inferiores a 7.000 €/año. Supone una alta penalización (15 pts).
 - **Baja (Umbral de pobreza):** Ingresos por unidad de consumo entre 7.000 € y 11.000 €/año. Supone una penalización moderada (7 pts).
 - **Media/Alta:** Ingresos por unidad de consumo superiores a 11.000 €/año. Ambas categorías se han unificado ya que no suponen un factor de riesgo económico y, por tanto, no suman puntos (0 pts).
- **Brecha Digital (Conectividad):** Sin acceso a Internet en el hogar (3 pts).
- **Brecha Digital (Hardware):** 0 ordenadores (5 pts), menos de 1 ordenador por hijo (2 pts).

Nota: Si la suma del riesgo socioeconómico supera los 16 puntos, la familia se considera en situación de alta vulnerabilidad.

5.2.2 Riesgo de abandono del alumno

Combina factores académicos, de apoyo y de arrastre socioeconómico para emitir una alerta crítica global:

- **Carga del Riesgo Socioeconómico:** De 16 a 21 puntos acumulados familiares (5 pts). Más de 21 puntos (10 pts).
- **Penalización Académica Temprana (1º o 2º de Primaria):** Los fracasos en ciclos iniciales marcan un déficit crónico. Suspense en evaluación regular (10 pts), asignatura suspensa a final de curso (20 pts), repetición del curso (40 pts).

- **Penalización Académica General (Resto de cursos):** Suspenso en evaluación regular (1 pt), asignatura suspensa a final de curso (5 pts), repetición (10 pts).
- **Necesidades de Apoyo y Adaptación:** Adaptación por motivos de discapacidad (10 pts). Adaptación por otros motivos o compensatoria (5 pts).

Estos valores numéricos quedan consolidados en *fact_rendimiento_anual*, lo que permite al Backend (FastAPI) entregar al Dashboard de forma instantánea el listado de centros con mayor concentración de alumnado en riesgo inminente de abandono.

Capa ETL con Apache Hop

EL éxito de un sistema de Business Intelligence radica en la calidad, consistencia y fiabilidad de sus datos. En esta fase central del proyecto se ha utilizado **Apache Hop**, una plataforma moderna e innovadora de orquestación y procesamiento de datos basada en metadatos, para diseñar e implementar los procesos de Extracción, Transformación y Carga (ETL).

Este capítulo detalla la arquitectura de los flujos de datos (*Pipelines*) y de trabajo (*Workflows*), exponiendo las decisiones técnicas tomadas para poblar el Datamart y calcular algorítmicamente los índices de riesgo de abandono escolar.

6.1 Configuración del entorno y conexiones

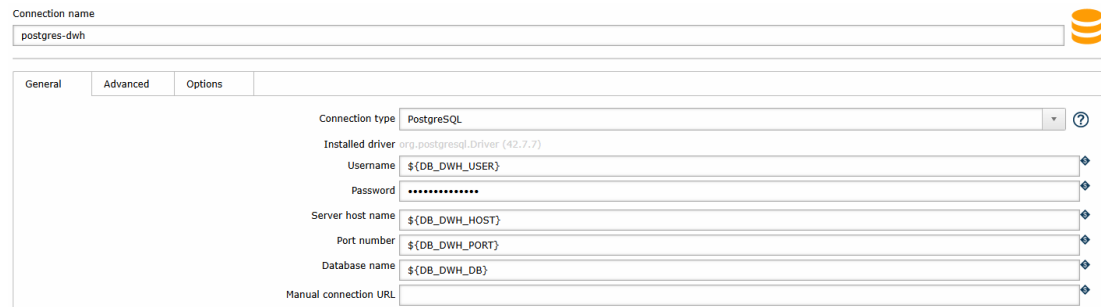
El ecosistema de Apache Hop se ha desplegado utilizando su imagen oficial de Docker (*apache/hop-web*), garantizando un entorno de ejecución inmutable. La parametrización de las conexiones a bases de datos se ha resuelto de forma dinámica utilizando variables de entorno de la JVM pasadas en el momento de arranque del contenedor (e.g., *-DDB_SOURCE_HOST*).

En la interfaz de Hop, se han configurado dos conexiones relacionales mediante el driver JDBC de PostgreSQL (ver Figura 6.1):

- **conn_source**: Conexión de solo lectura apuntando a la base de datos transaccional simulada.
- **conn_dwh**: Conexión de lectura y escritura apuntando al esquema dimensional del Data Warehouse.

6.2 Análisis de pipelines

A continuación, se detalla el ciclo de vida del dato desglosando el funcionamiento técnico de cada uno de los pipelines (*.hpl*) que componen el sistema ETL. Un pipeline en Apache Hop



The screenshot shows the 'General' tab of the 'postgres-dwh' connection configuration in Apache Hop. The 'Connection type' is set to 'PostgreSQL'. The 'Installed driver' is 'org.postgresql.Driver (42.7.7)'. The 'Username' is '\$(DB_DWH_USER)', 'Password' is masked with dots, 'Server host name' is '\$(DB_DWH_HOST)', 'Port number' is '\$(DB_DWH_PORT)', and 'Database name' is '\$(DB_DWH_DB)'. The 'Manual connection URL' field is empty. The Apache Hop logo is visible in the top right corner.

Figura 6.1: Configuración de la conexión *postgres-dwh*.

es un grafo dirigido acíclico (DAG) donde los nodos (*Transforms*) ejecutan operaciones específicas (lectura, filtrado, cálculos) y las aristas (*Hops*) canalizan el flujo de registros directamente en memoria.

6.2.1 Volcado a staging

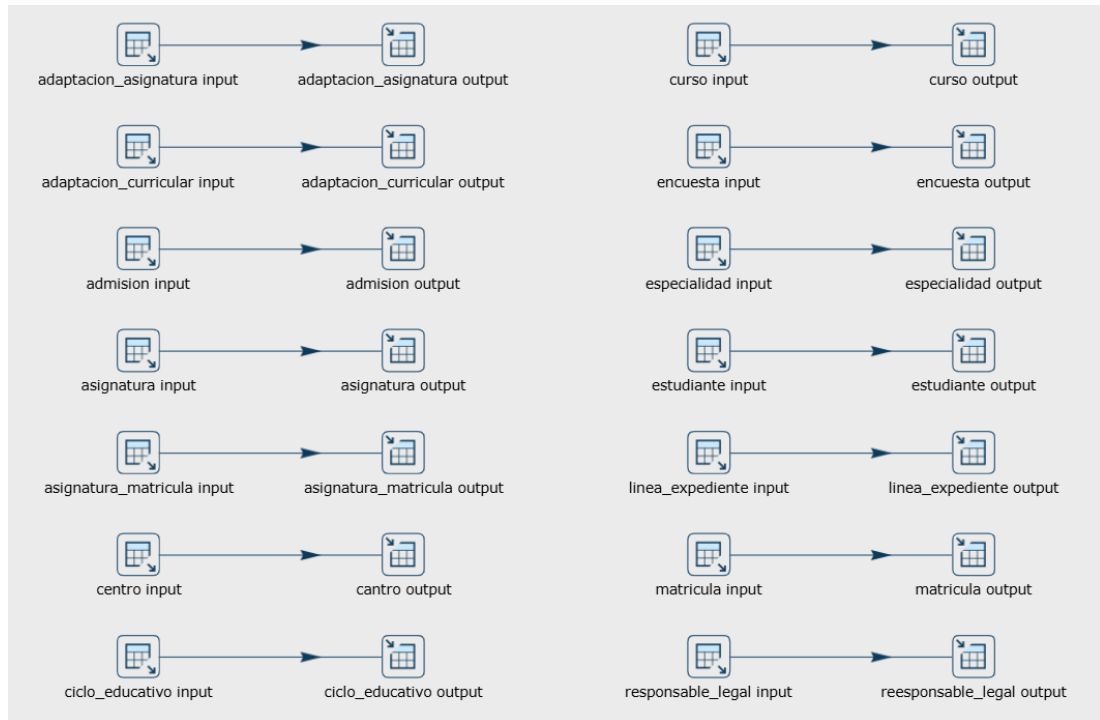
Para no sobrecargar la base de datos origen transaccional durante los cruces analíticos posteriores, el primer paso estricto de la arquitectura ETL consiste en realizar una clonación exacta de las tablas de producción hacia el esquema intermedio transitorio *raw* (Staging Area).

Como se aprecia en la Figura 6.2, el pipeline *extract_all_to_staging.hpl* establece 14 hilos independientes de tipo *Table Input* (origen) conectados a *Table Output* (destino). Este componente implementa un mecanismo de extracción incremental mediante el uso del parámetro dinámico `$(CURSO_ACTUAL)`. Los nodos de extracción ejecutan subconsultas SQL que filtran y capturan exclusivamente los datos del curso lectivo en vigor (ej. `WHERE curso_academico = '$(CURSO_ACTUAL)'`). Para permitir recargas históricas completas, se incluye la condición lógica `OR '$(CURSO_ACTUAL)' = ''`, garantizando una lectura masiva si el orquestador omite el parámetro. Este diseño evita volcados completos innecesarios y minimiza drásticamente la carga de red en cada ejecución por evaluación.

Una decisión de diseño fundamental documentada en este paso es la **ausencia de claves foráneas** en el esquema destino. Al no existir restricciones referenciales en la capa *raw*, Hop puede paralelizar la inserción masiva de todas las entidades simultáneamente sin generar bloqueos lógicos por dependencias, minimizando drásticamente la ventana de tiempo de extracción.

6.2.2 Dimensión de demografía familiar

El núcleo de las reglas de negocio del proyecto reside en este pipeline. Su misión es ingerir los datos brutos de la tabla de encuestas (*raw.encuesta*) y destilarlos para construir una dimensión de baja cardinalidad que consolide los perfiles de riesgo socioeconómico. Su imple-

Figura 6.2: Flujo del pipeline *extract_all_to_staging.hpl*.

mentación técnica encadena operaciones aritméticas, lógicas condicionales y deduplicación (ver Figura 6.3):

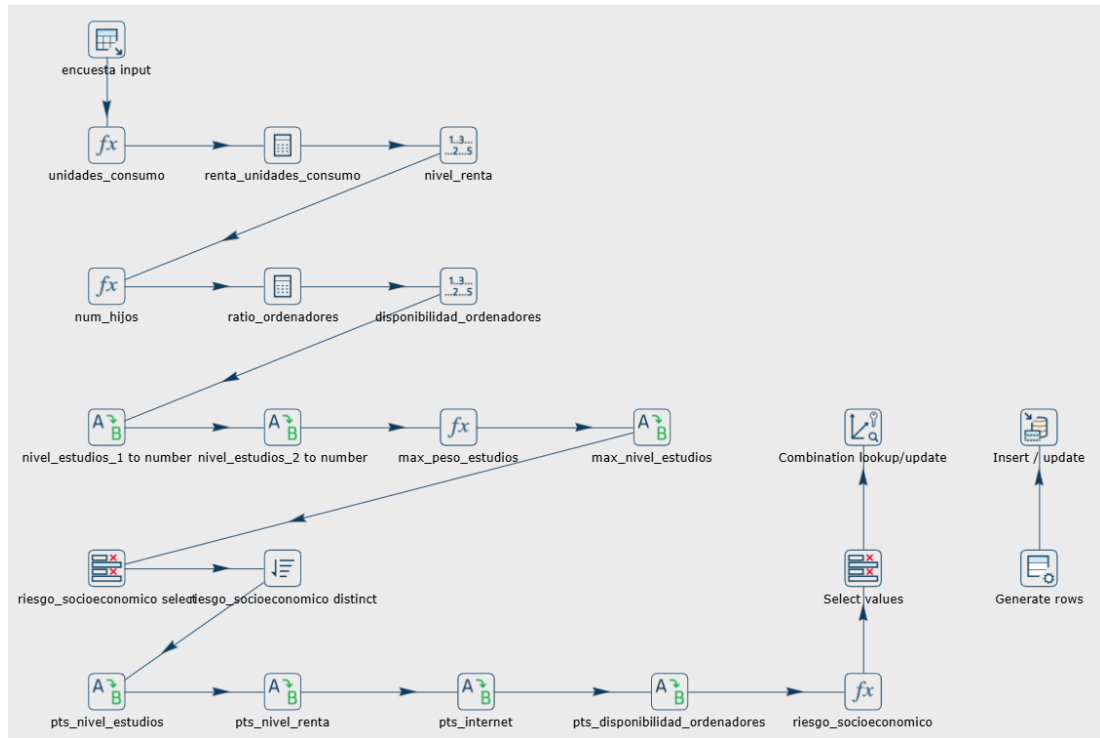
1. **Cálculo de Hijos a Cargo (*Formula*):** Dado que la tabla origen *raw.encuesta* solo informa del número total de integrantes del hogar, el pipeline calcula el número de hijos restando los adultos. Asume 1 adulto si no hay datos del segundo tutor, o 2 si los hay, garantizando matemáticamente un mínimo de 1 hijo.
2. **Cálculos de Ratio y Consumo (*Calculator*):** Se inicializan variables continuas dividiendo algebraicamente los ingresos brutos entre las unidades de consumo del hogar, obteniendo la *renta_unidades_consumo*. Paralelamente, se divide *num_ordenadores* entre *num_hijos* calculados en el paso anterior para obtener la *ratio_ordenadores*.
3. **Discretización de Variables Continuas (*NumberRange*):** Las variables calculadas se segmentan en bandas analíticas fijas:
 - *nivel_renta*: "Muy baja" (< 7.000 €), "Baja" (7.000 € - 11.000 €) o "Media/Alta" (> 11.000 €).
 - *disponibilidad_ordenadores*: "Ninguno" (0), "Menos de 1 por hijo" (0.001 a 0.999), o "1 o mas por hijo" (≥ 1).

4. **Máximo Nivel de Estudios (*ValueMapper* y *Formula*):** El pipeline transforma los niveles de estudio de ambos progenitores en un peso numérico (Universitarios=3, Secundarios=2, Primarios=1, Sin Estudios=0), aplica una función $\text{MAX}([\text{peso_estudios_1}], [\text{peso_estudios_2}])$ para quedarse con el nivel más alto del hogar, y vuelve a traducir ese número a cadena de texto (*max_nivel_estudios*).
5. **Asignación de Puntos de Riesgo (*ValueMapper*):** Se inyectan los pesos definidos por el negocio para calcular el riesgo de abandono:
 - *pts_nivel_renta*: Muy baja (15 pts), Baja (7 pts), Media/Alta (0 pts).
 - *pts_nivel_estudios*: Sin Estudios (10 pts), Primarios (6 pts), Secundarios (2 pts), Universitarios (0 pts).
 - *pts_disponibilidad_ordenadores*: Ninguno (5 pts), Menos de 1 (2 pts), 1 o más (0 pts).
 - *pts_internet*: Sin acceso (3 pts), Con acceso (0 pts).
6. **Ecuación Maestra (*Formula*):** Se materializa la variable predictiva final sumando todos los factores: $[\text{pts_disponibilidad_ordenadores}] + [\text{pts_internet}] + [\text{pts_nivel_estudios}] + [\text{pts_nivel_renta}]$.
7. **Optimización de Cardinalidad (*SortRows* - *unique_rows=Y*):** Para evitar que la dimensión crezca con cada estudiante, un nodo ordena por los atributos cualitativos y descarta los duplicados exactos en memoria. De miles de encuestas, el Data Warehouse solo almacena las combinaciones únicas posibles.
8. **Carga en Datamart (*Combination lookup/update*):** Se vuelcan los perfiles deduplicados en *dwh.dim_demografia_familiar*, generando una clave subrogada autoincremental (*id_demografia_familiar*) que será utilizada posteriormente por la tabla de hechos.

6.2.3 Dimensión de estudiante

La dimensión del alumnado requiere un modelado avanzado, ya que la casuística de un estudiante no es estática: puede cambiar de domicilio, de tutor legal o de situación familiar a lo largo de los años. Si el sistema simplemente sobrescribiese estos datos, se perdería el contexto socioeconómico real que tenía el alumno cuando obtuvo una calificación pasada. Para solventarlo, este pipeline implementa un algoritmo SCD (Slowly Changing Dimension) Tipo 2 apoyado en la limpieza previa de cadenas (ver Figura 6.4).

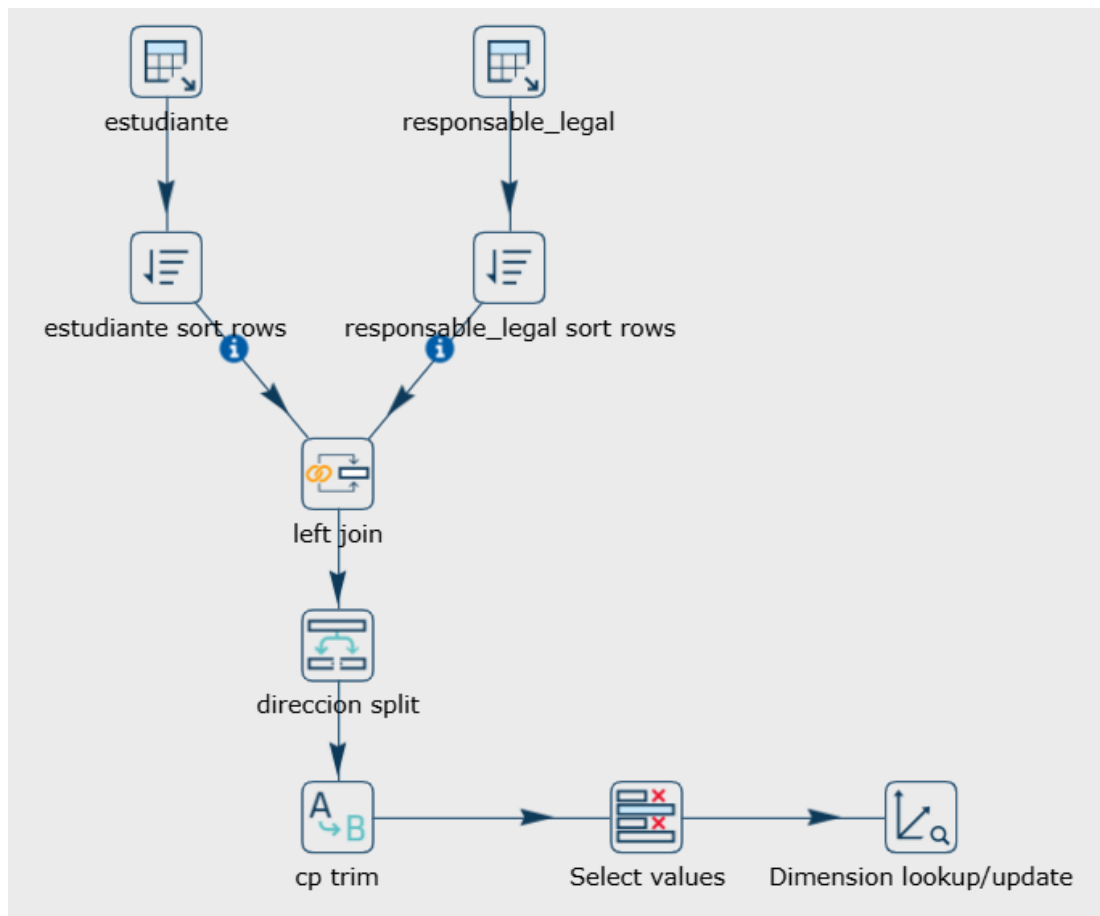
1. **Cruce y Limpieza de Cadenas (*MergeJoin* y *FieldSplitter*):** El flujo extrae asincrónicamente las entidades *raw.estudiante* y *raw.responsable_legal*, ordenándolas en memoria

Figura 6.3: Flujo del pipeline *dim_demografia_familiar.hpl*.

(*SortRows*) para ejecutar un *Merge Join* tipo LEFT OUTER por el campo *cod_responsable*. A continuación, un nodo *FieldSplitter* disecciona la cadena cruda de la dirección separándola por comas en las columnas *calle_portal*, *puerta*, *cp* y *ciudad*.

2. **Casteo de Datos (*ReplaceString* y *SelectValues*):** El nodo *ReplaceString* elimina la subcadena "CP " de los códigos postales, permitiendo que el paso subsiguiente (*SelectValues*) lo formatee analíticamente como un tipo numérico (Integer) y renombre simultáneamente las variables de los tutores a la nomenclatura final del DWH.
3. **Algoritmo SCD Tipo 2 (*Dimension lookup/update*):** El motor principal del pipeline es este nodo nativo de Hop, configurado para rastrear el ciclo de vida del *num_expediente* (Clave Natural) gestionando automáticamente una clave subrogada *id_estudiante* (autoincremental), un contador de *version* y las marcas temporales *date_from* y *date_to*. La maestría de esta configuración reside en la micro-gestión de actualizaciones campo a campo:
 - **Punch through (Comportamiento SCD Tipo 1):** Campos intrínsecos como el *dni*, *nombre* o *fecha_nacimiento*. Una corrección en origen de estos datos no justifica crear un nuevo histórico; Hop sobrescribe el valor directamente en todas las versiones pasadas de ese estudiante.

- **Insert (Comportamiento SCD Tipo 2):** Campos dependientes del contexto socioeconómico, como *cod_responsable*, *calle_portal*, *ciudad* y *monoparental*. Si Hop detecta una mutación en cualquiera de estos campos frente al registro vigente en el DWH, congela la fila antigua (estampando el *date_to* actual), e inserta una nueva fila virgen con la versión incrementada y un nuevo *id_estudiante*.
- **Update:** Datos de contacto secundarios como el *email*, que se actualizan silenciosamente en la versión activa sin generar ruido histórico.

Figura 6.4: Flujo del pipeline *dim_estudiante.hpl*.

6.2.4 Dimensión de tiempo

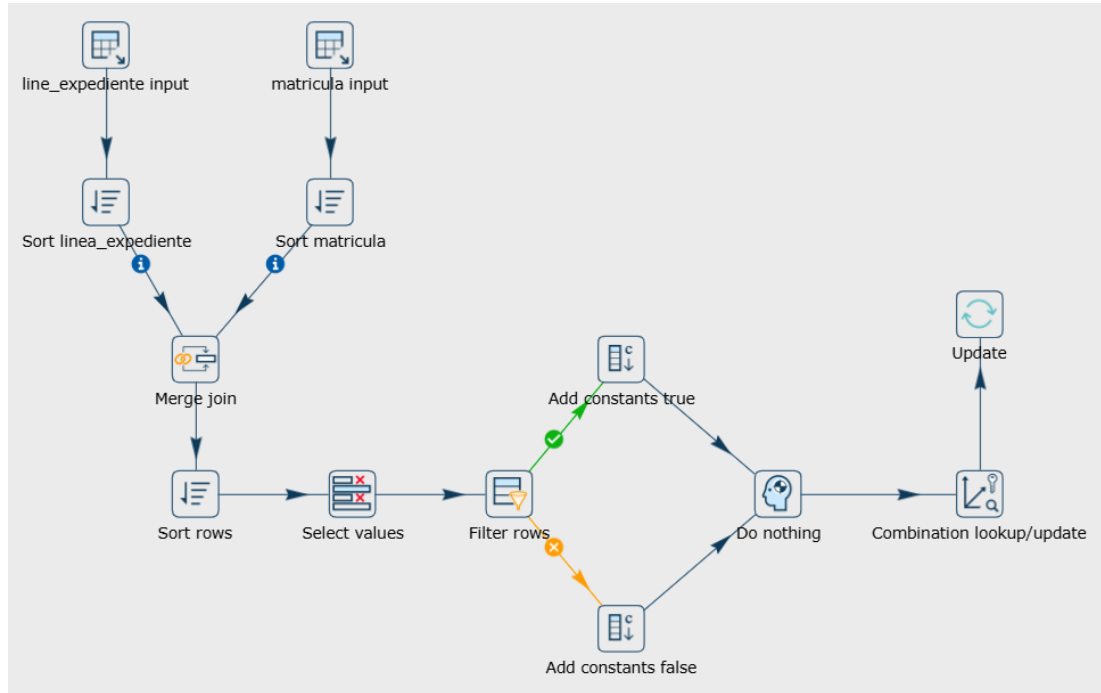
A diferencia de un calendario estándar por días o meses, este sistema requiere una dimensión de tiempo puramente académica que categorice los momentos evaluativos de un alumno (por ejemplo, "Curso 2023-2024, 1ª Evaluación"). Este pipeline infiere automáticamente estas temporalidades desde los propios registros transaccionales (ver Figura 6.5).

1. **Cruce Transaccional (*MergeJoin*):** El flujo no lee de una tabla maestra explícita, sino que extrae las operaciones transaccionales *raw.linea_expediente* y *raw.matricula*. Tras ordenar ambas por *cod_matricula* mediante nodos *SortRows*, aplica un cruce de tipo INNER JOIN para asociar la *evaluacion* en la que se puso cada nota con el *curso_academico* de la matrícula correspondiente.
2. **Extracción del Dominio (*SortRows* - *unique_rows=Y*):** De los cientos de miles de líneas de expediente cruzadas, se extrae el dominio temporal real filtrando únicamente los pares únicos de *curso_academico* y *evaluacion*, purificando así la dimensionalidad.
3. **Análisis Léxico Condicional (*FilterRows*):** Un paso crítico para el negocio es aislar si una evaluación es la definitiva o una parcial (fundamental para el pipeline posterior de *fact_rendimiento_anual*). Un nodo *FilterRows* analiza léxicamente el texto de la evaluación buscando la subcadena "Final" (evaluando múltiples casuísticas mediante condicionales *CONTAINS* encadenados con operadores OR: "Final", "final", "FINAL").
4. **Bifurcación de Flujo Dirigido:** Dependiendo del resultado del filtro anterior, Hop enruta el dato físicamente por dos caminos paralelos. Si el filtro resulta verdadero, el registro se desvía a un nodo *Add constants true* que inyecta la variable booleana (*is_final* = Y). Si es falso, viaja por la rama inferior a *Add constants false* (*is_final* = N). Ambas ramas se reconcilian posteriormente en un nodo *Dummy* integrador.
5. **Carga Híbrida (*Combination lookup* + *Update*):** El pipeline implementa un patrón avanzado de persistencia en Hop. Primero, el nodo *Combination lookup/update* consolida las claves (*curso_academico* y *evaluacion*) e inserta la fila generando automáticamente la clave subrogada *id_tiempo*. Como este paso nativo no admite insertar atributos no-clave simultáneamente, el flujo encadena de forma síncrona un paso *Update*. Este busca el *id_tiempo* recién instanciado y le hace un UPDATE para persistir el metadato booleano *is_final* en la base de datos de destino.

6.2.5 Dimensión de asignatura

El modelo de asignaturas en el sistema origen transaccional presenta una arquitectura relacional altamente normalizada (en "Copo de Nieve"). Para modelarlo en el Data Warehouse bajo un esquema de Estrella óptimo para lectura, este pipeline colapsa jerarquías relacionales en una única fila dimensional (ver Figura 6.6).

1. **Ingesta y Búsquedas en Cascada (*DBLookup*):** Hop extrae *raw.asignatura* y, mediante tres nodos *DBLookup* encadenados, recorre las claves foráneas hacia arriba. Primero recupera el *num_optativas* desde la tabla *raw.curso*; después, obtiene la *especialidad*

Figura 6.5: Flujo del pipeline *dim_tiempo.hpl*.

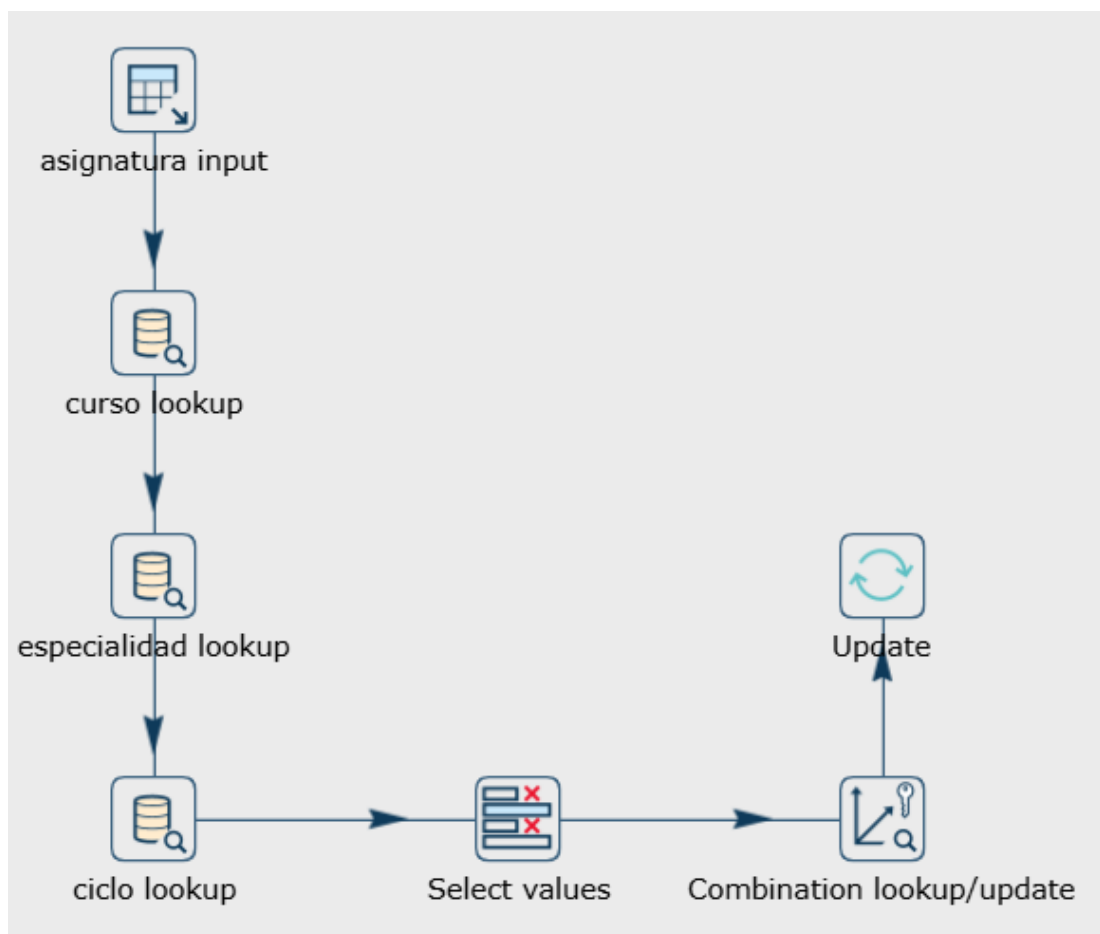
y su ciclo asociado desde *raw.especialidad*; y finalmente, extrae el marco normativo (*real_decreto*, *decreto_autonomico*) desde *raw.ciclo_educativo*.

2. **Consolidación (*SelectValues* y *Combination lookup*):** Una vez que todas las ramas jerárquicas han sido aplanadas en un único vector, se purgan las claves intermedias y se genera el registro en la dimensión destino con su *id_asignatura* correspondiente.

6.2.6 Dimensión de curso

Esta dimensión no solo almacena los metadatos del curso y ciclo, sino que alberga una regla de negocio fundamental: determinar la edad teórica esperada para un alumno en cada nivel, lo cual permitirá posteriormente a la tabla de hechos calcular el desfase de edad (indicador clave de repetición y riesgo académico, ver Figura 6.7).

1. **Mapeo de Nivel Educativo (*ValueMapper*):** Tras aplanar el curso, la especialidad y el ciclo educativo, un nodo inyecta paramétricamente la edad teórica de acceso al ciclo mediante pares clave-valor (ej. "PRI" → 5 años, "ESO" → 11 años, "BAC" → 15 años).
2. **Cálculo Algebraico de Edad Ideal (*Calculator*):** Un nodo matemático suma dinámicamente el número ordinal del curso a la edad base del ciclo. Así, un alumno en 3º

Figura 6.6: Flujo del pipeline *dim_asignatura.hpl*.

de la ESO (ciclo base 11) tiene una *edad_ideal* de 14 años. Este valor precalculado se consolida permanentemente en *dim_curso*.

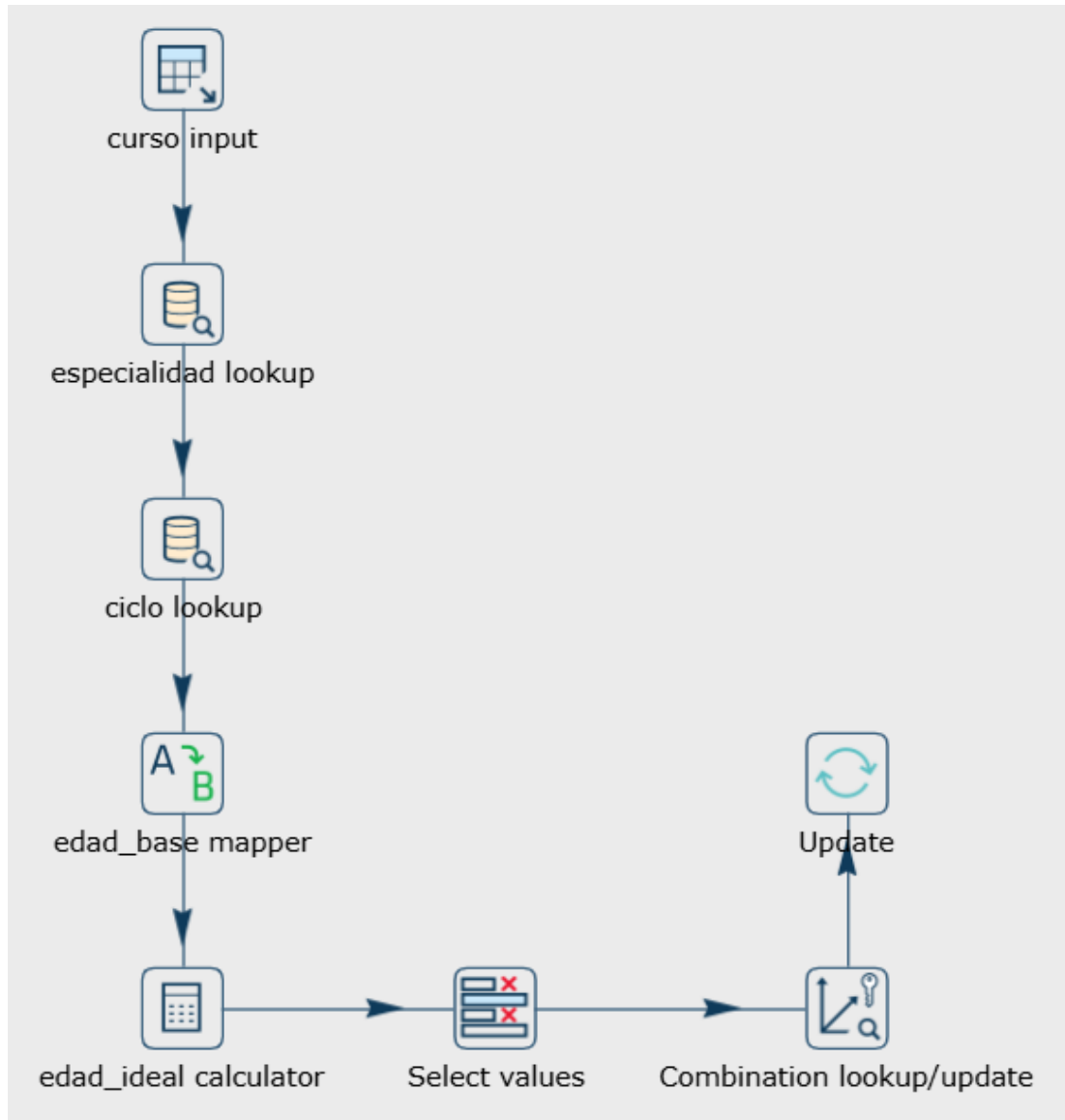
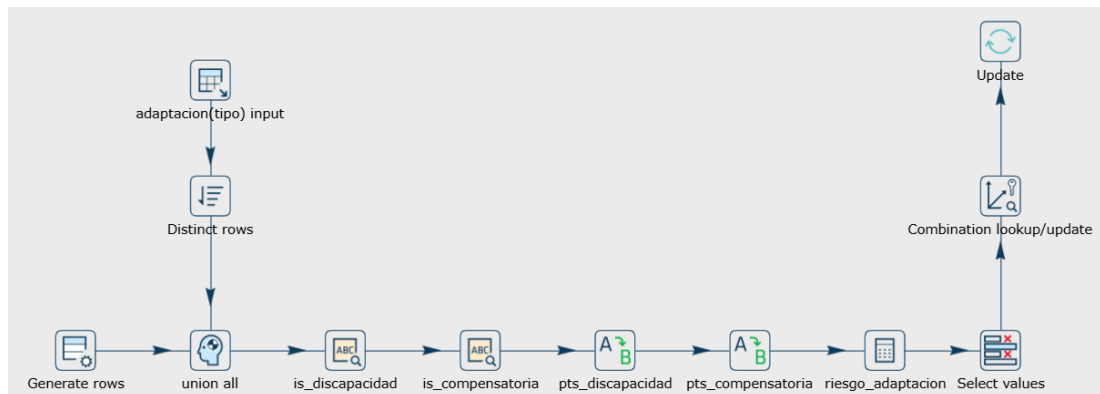


Figura 6.7: Flujo del pipeline *dim_curso.hpl*.

6.2.7 Dimensión de adaptación

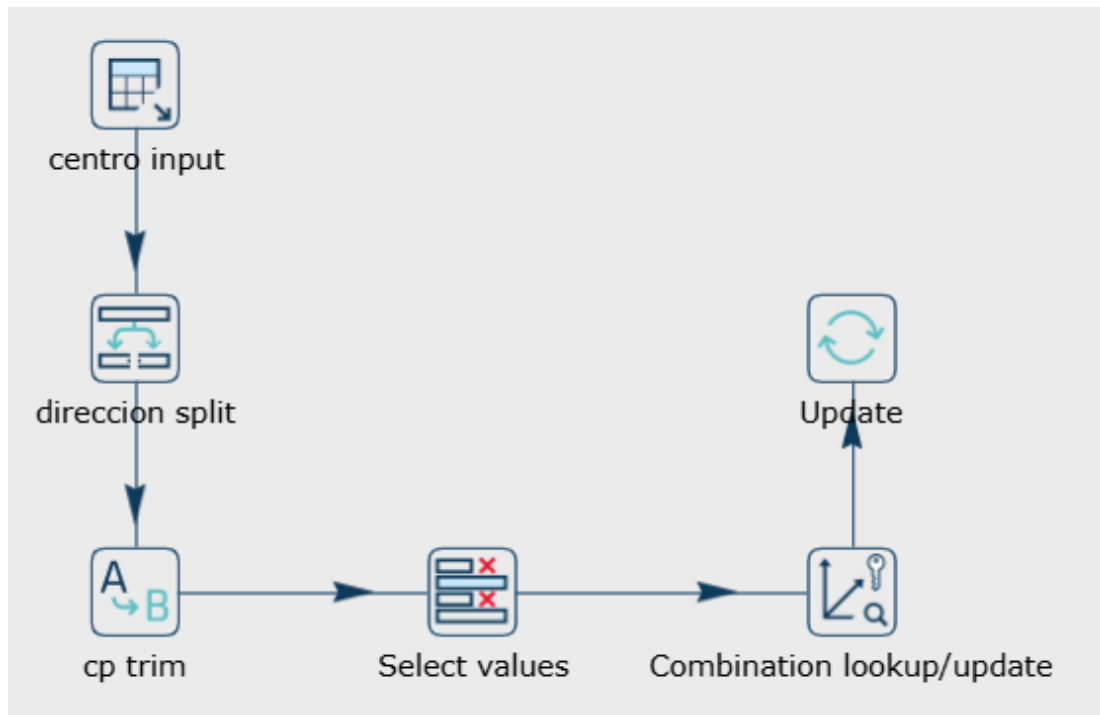
El registro transaccional de adaptaciones curriculares en los centros suele ser un campo de texto libre con la patología o motivo de la adaptación, dificultando su explotación analítica. Este pipeline actúa como un motor de extracción de reglas clínicas (ver Figura 6.8).

1. **Limpieza de Dominio (*SortRows* y *RowGenerator*):** Dado que la mayoría del alumnado no presenta adaptaciones, el flujo genera sintéticamente una fila "Ninguna" para asegurar la integridad referencial en el modelo de estrella, y posteriormente extrae la lista de tipos únicos registrados en el histórico.
2. **Clasificación Clínica por Expresiones Regulares (*RegexEval*):** Dos nodos paralelos aplican potentes patrones Regex sobre el texto crudo:
 - ***is_discapacidad*:** Busca que contenga (TND, Discalculia, Dislexia, Motórica, Visual, Asma, Mutismo, TDAH, TEA) para identificar discapacidades fisiológicas o neuropsicológicas.
 - ***is_compensatoria*:** Rastrea que incluya (Absentismo, Ludopatía, Acogimiento, Convivencia, Ciberacoso, Conducta, Intercultural, Social, Inestabilidad, Inmersión, Lingüística) para aislar intervenciones estrictamente sociológicas.
3. **Ponderación de Severidad (*ValueMapper* y *Calculator*):** Cada tipología desencadena un peso distinto (10 puntos para las fisiológicas, 5 puntos para las sociológicas). Un nodo matemático suma los puntos creando la métrica unificada *riesgo_adaptacion*.

Figura 6.8: Flujo del pipeline *dim_adaptacion.hpl*.

6.2.8 Dimensión de centro

El pipeline del centro educativo, aunque estructuralmente sencillo al procesar un catálogo estático y reducido de registros, encapsula lógicas de limpieza para la geolocalización analítica. El campo crudo *direccion* se somete a un nodo *FieldSplitter* que particiona los componentes por su coma delimitadora (en las columnas Calle, CP, y Ciudad), se limpia iterativamente la cadena del código postal eliminando el prefijo literario "CP " (*ReplaceString*), y se fuerza su formateo en base de datos al tipo nativo Integer en un paso *SelectValues* previo a la generación de la clave autoincremental *id_centro* (ver Figura 6.9).

Figura 6.9: Flujo del pipeline *dim_centro.hpl*.

6.2.9 Tabla de hechos de calificaciones

Este es el pipeline central del sistema. Su objetivo es poblar la tabla de hechos atómica *fact_calificaciones*, cruzando los eventos transaccionales con todas las dimensiones construidas previamente y calculando métricas volátiles en el momento del evento (ver Figuras 6.10 y 6.11).

1. **Ingesta y Cruce Base (Table Input, SortRows, DBLookup):** El flujo arranca extrayendo las calificaciones desde *raw.linea_expediente*, ordenándolas en memoria, y utilizando un nodo *DBLookup* ("matricula lookup") contra *raw.matricula* para heredar metadatos vitales como la *fecha* formal de matriculación, el *curso_academico* y el *cod_centro*.
2. **Resolución de Claves Estáticas (DBLookup en Cascada):** A continuación, se encadenan cuatro nodos idénticos de tipo *DBLookup* que interrogan a las dimensiones estáticas (*dim_asignatura*, *dim_centro*, *dim_curso* y *dim_tiempo*) cotejando las claves naturales para extraer sus identificadores subrogados (*id_asignatura*, etc.). Críticamente, del nodo "dim_curso lookup" no solo se extrae la clave, sino que se arrastra la variable precalculada *edad_ideal*.
3. **Gestión de Nulos en Adaptaciones (StreamLookup e IfNull):** Para resolver la dimensión de adaptaciones curriculares, el flujo lee asincrónicamente las tablas *raw* de

adaptaciones, hace un *StreamLookup* en memoria, e implementa un nodo *IfNull* programado para inyectar la cadena "Ninguna" si el alumno no presenta patologías registradas. Un *DBLookup* posterior extrae su *id_adaptacion*.

4. **Resolución Temporal SCD2 (*Dimension Lookup*):** Para enlazar la dimensión del estudiante, se emplea un potente nodo *Dimension Lookup* pasándole la *fecha* de matrícula extraída en el primer paso. Hop coteja automáticamente esta fecha contra los atributos *date_from* y *date_to* de la dimensión, devolviendo el *id_estudiante* que encapsula la realidad familiar que tenía el alumno **en el momento exacto de matricularse**.
5. **Cálculo Algebraico de Retraso Académico (*StringCut* y *Calculator*):** Un nodo *StringCut* ("*año_curso cut*") extrae los 4 primeros dígitos de la cadena *curso_academico* (ej. "2023" de "2023-2024"), y un paso *SelectValues* lo formatea a numérico. Posteriormente, tres nodos *Calculator* entran en juego: extraen el año de la fecha de nacimiento con *YEAR_OF_DATE*, restan ambos años para obtener la *edad_academica* del alumno, y le restan la *edad_ideal* arrastrada de la dimensión curso para obtener matemáticamente la métrica *desfase_edad*.
6. **Recálculo Demográfico Histórico (*DBJoin* y *Combination Lookup*):** El paso más exigente. Un nodo *DBJoin* interroga la tabla de encuestas inyectando la restricción SQL *fecha <= ?* (usando la fecha de la matrícula) para rescatar la encuesta vigente en aquel momento. A partir de sus columnas crudas, una decena de nodos recalculan las unidades de consumo (*Formula*), la renta por consumo (*Calculator*), categorizan los ingresos (*NumberRange*), discretizan el nivel de estudios (*ValueMapper* y *Formula*) y suman todos los puntos para inferir el *riesgo_socioeconomico*. Finalmente, el nodo avanzado *CombinationLookup* mapea estos atributos contra *dim_demografia_familiar* para asignar o crear el *id_demografia_familiar*.
7. **Carga Incremental tipo Upsert (*Insert / update*):** Superada la lógica de transformación, el pipeline utiliza un nodo transaccional *Insert / update* para volcar los datos en el esquema destino. Este componente evalúa en tiempo real si la combinación de claves ya existe en la tabla de hechos. Si existe, ejecuta una actualización (*Update*) sobre las métricas; si es un registro inédito, realiza una inserción limpia (*Insert*). Este mecanismo asegura la persistencia del historial acumulado y es el pilar que da soporte a la arquitectura *Change Data Capture (CDC)* de la solución.

6.2.10 Tabla de hechos de rendimiento por evaluación

El último proceso del ecosistema ETL actúa como el motor predictivo del proyecto, consolidando las calificaciones para calcular el riesgo final de abandono escolar. Para dotar al sis-

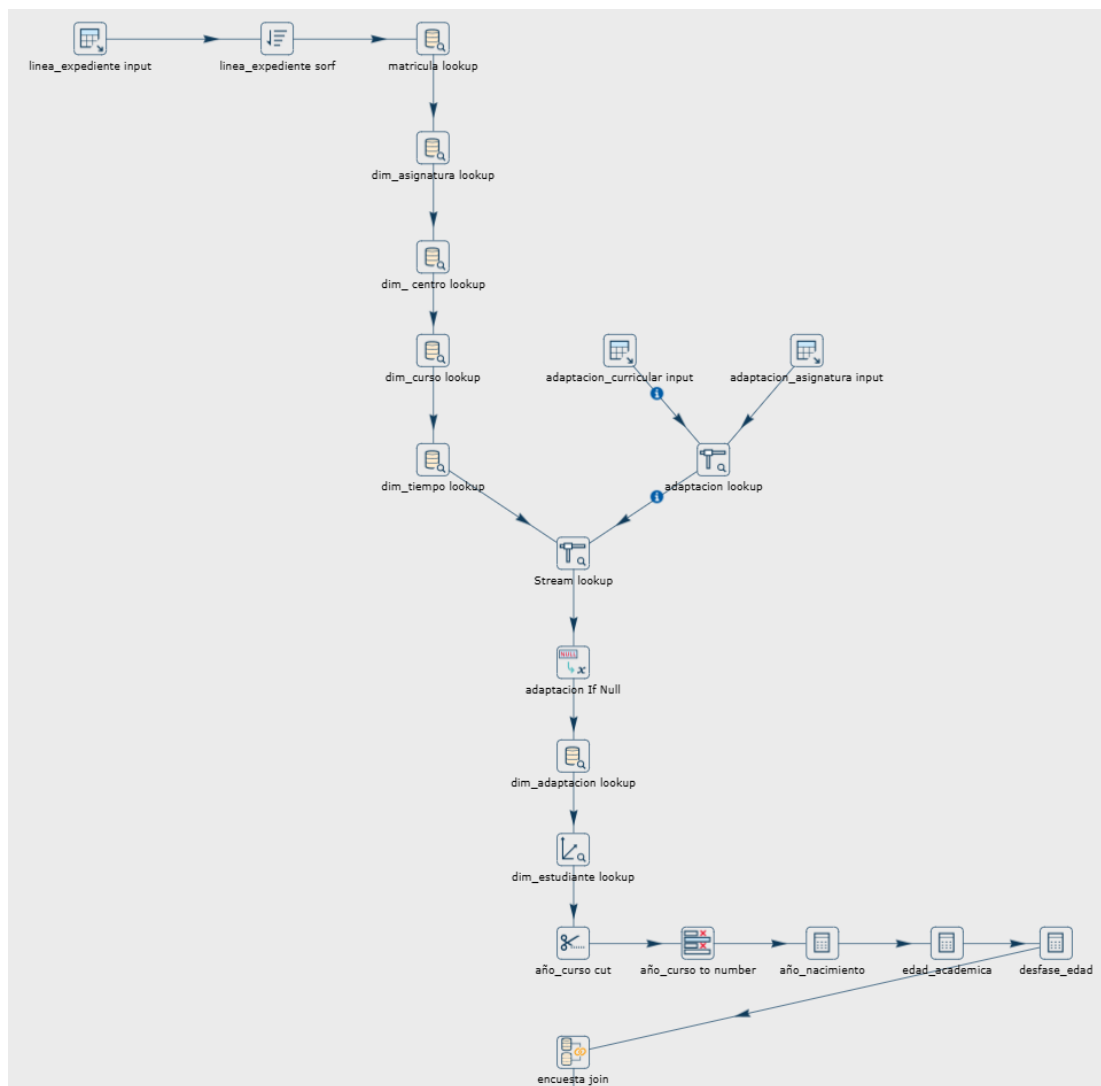


Figura 6.10: Flujo del pipeline *fact_calificaciones.hpl* (Parte 1).

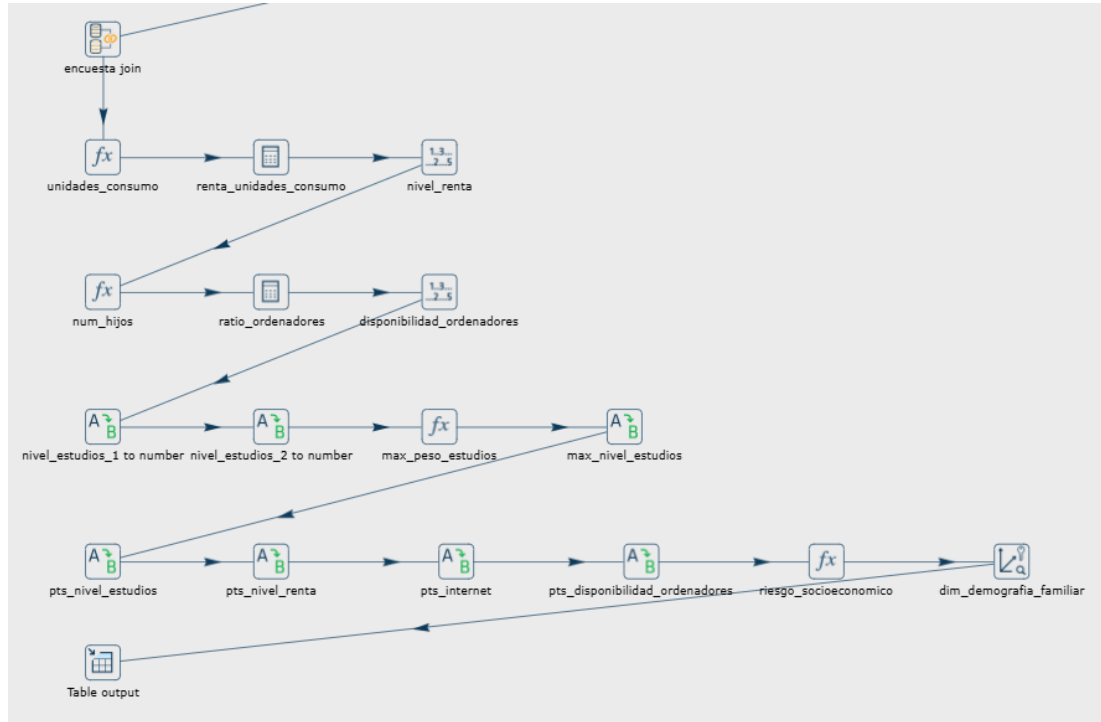


Figura 6.11: Flujo del pipeline *fact_calificaciones.hpl* (Parte 2).

tema de la capacidad de procesar recargas históricas completas y, al mismo tiempo, soportar un modo de ejecución por evaluación de alta eficiencia, este cálculo se ha dividido en dos pipelines paralelos: *fact_rendimiento_anual_full.hpl* y *fact_rendimiento_anual_incremental.hpl*.

Lógica de consolidación masiva (Pipeline Full)

Este pipeline (ver Figuras 6.12 y 6.13) abandona la clásica consolidación estática anual y adopta una arquitectura de "Suma Acumulada" manteniendo el grano de la tabla a nivel de Evaluación (*id_estudiante* × *id_tiempo*).

1. **Ingesta y Tipificación Analítica (Table Input, DBLookup y Formula):** El flujo ingiere *dwh.fact_calificaciones* y cruza mediante *DBLookup* con todas las dimensiones clave (*dim_tiempo*, *dim_curso*, *dim_demografia_familiar*, *dim_adaptacion*). Tras esto, un nodo *Formula* bautizado como "flags analíticos" perfila las casuísticas de cada nota. Es aquí donde se detecta de forma temprana si el registro pertenece al ciclo crítico de 1° y 2° de Primaria (*is_prim_1_2*), y se despliegan vectores binarios para segmentar si un suspenso es de evaluación final o parcial, y si pertenece a dicho ciclo crítico o al resto.
2. **Rastreo Histórico Acumulado (Flujo Principal):** Tras una ordenación cronológica estricta (*id_estudiante* × *curso_academico* × *orden_evaluacion*), un nodo *Group By*

agrupa por estudiante activando la directiva *"Include all rows"*. Implementando funciones *Cumulative sum* sobre los vectores previamente generados, el sistema emula una *Window Function*. Esto arrastra en el registro actual todo el bagaje histórico de suspensos segmentados (parciales/finales y primaria/resto).

3. **Motor Detección de Repetidores (Sub-rama Paralela):** El flujo se bifurca hacia una rama auxiliar especializada en trazar las repeticiones. Puesto que los parciales duplican los cursos, un nodo *Group By* colapsa la granularidad a un solo registro por año. Posteriormente, un nodo *Analytic Query* (LAG 1) lee el *curso_anterior* del alumno. Un nodo *Formula* detecta matemáticamente la repetición y separa mediante *flags* si esta se produce en 1º/2º de Primaria o en cursos superiores. Finalmente, un segundo *Group By* con sumatoria acumulativa calcula el cómputo histórico de repeticiones del alumno (*num_repetir_1_2_pri* y *num_repetir_resto*).
4. **Fusión y Reencuentro (StreamLookup):** Tras calcular la historia de repeticiones de la sub-rama, se hace un cruce (Left Join en memoria) mediante el nodo *StreamLookup*. Este vuelve a inyectar al flujo principal transaccional toda la mochila de repeticiones basándose en la clave *id_estudiante × curso_academico*.
5. **Ecuación Maestra de Riesgo (Formula):** Un potente nodo de fórmulas se encarga de convertir todas las acumulaciones en puntuaciones predictivas (Scoring). El **Riesgo Académico** se calcula ponderando de manera asimétrica los factores: suspensos finales en 1º/2º Primaria (20 pts), suspensos parciales (10 pts) y penalizando drásticamente la repetición en este ciclo temprano (40 pts), frente a las métricas del resto de cursos que tienen un peso menor. A la par, evalúa el riesgo base familiar creando una puntuación escalonada para el **Riesgo Socioeconómico** (10 pts si supera el umbral severo, 5 si es riesgo moderado).
6. **Cálculo del Indicador de Abandono (Calculator):** Como culminación, un nodo sumatorio simple (*ADD3*) consolida matemáticamente el *riesgo_academico*, el *riesgo_adaptacion* y los puntos socioeconómicos. Esto arroja el indicador definitivo *riesgo_abandono*, que finalmente es almacenado en la tabla *dwh.fact_rendimiento_anual* a través de un nodo masivo *Table Output*.

Adaptaciones del pipeline Incremental

El principal problema arquitectónico del flujo *Full* es que, para calcular la penalización por fracasos previos, necesita volcar en la memoria RAM el historial completo de calificaciones de toda la comunidad autónoma. En una carga inicial esto es necesario, pero en una recarga periódica de evaluación resulta insostenible por el enorme consumo de recursos y tiempo. Para re-

solver esta ineficiencia matemática, se diseñó la variante *fact_rendimiento_anual_incremental.hpl* (ver Figura 6.14), que aplica las siguientes soluciones:

1. **Ingesta Parametrizada y Recálculo del Curso:** El nodo *Table Input* inyecta la variable dinámica $\{CURSO_ACTUAL\}$, leyendo exclusivamente las calificaciones del ciclo lectivo en vigor. Se opta por recalcular el curso actual completo (en lugar de inyectar solo una única evaluación) para poder procesar y solventar automáticamente cualquier rectificación en las notas producida antes del cierre de actas, manteniendo un volumen de datos mínimo comparado con la carga histórica total.
2. **Recuperación del Historial (*Database Join*):** Al no disponer del bagaje histórico en RAM, el nodo *Group By* (Suma Acumulada) se vuelve inoperante a nivel interanual. Para solucionarlo, un nodo *Database Join* interroga directamente a la tabla destino (*fact_rendimiento_anual*) filtrando por el año anterior del alumno (ver detalle en la Figura 6.15). El objetivo clave es recuperar su *riesgo_academico* ya consolidado. Dado que el modelo matemático del riesgo es puramente aditivo, el sistema calcula de forma aislada los puntos de los suspensos del año actual y simplemente los suma al riesgo acumulado extraído de la base de datos.
3. **Detección de Repetidores Dinámica:** La sub-rama analítica basada en LAG 1 se descarta al carecer del año previo en memoria. El flujo aprovecha el cruce anterior con la propia tabla *fact_rendimiento_anual* para consultar si el alumno arrastra repeticiones previas (*num_repetir_1_2_pri* y *num_repetir_resto*), integrando estas penalizaciones en la fórmula actual de manera directa.
4. **Carga no Destructiva (*Insert / update*):** El nodo final abandona el volcado masivo y destructivo (*Table Output*). Utiliza en su lugar el componente transaccional *Insert / update*, permitiendo inyectar los nuevos cálculos de riesgo en el Data Warehouse mientras preserva intacto todo el historial de los años anteriores.

6.3 Orquestación y workflows

Todos estos pipelines independientes (*.hpl*) son llamados y secuenciados por flujos de trabajo o *Workflows* (*.hwf*). Mientras que los pipelines manejan registros fila a fila, los workflows manejan la lógica de control a nivel de proceso.

La orquestación del proyecto sigue una arquitectura jerárquica encabezada por el flujo principal *main.hwf* (ver Figura 6.16), el cual delega la carga en dos sub-workflows de especialidad:

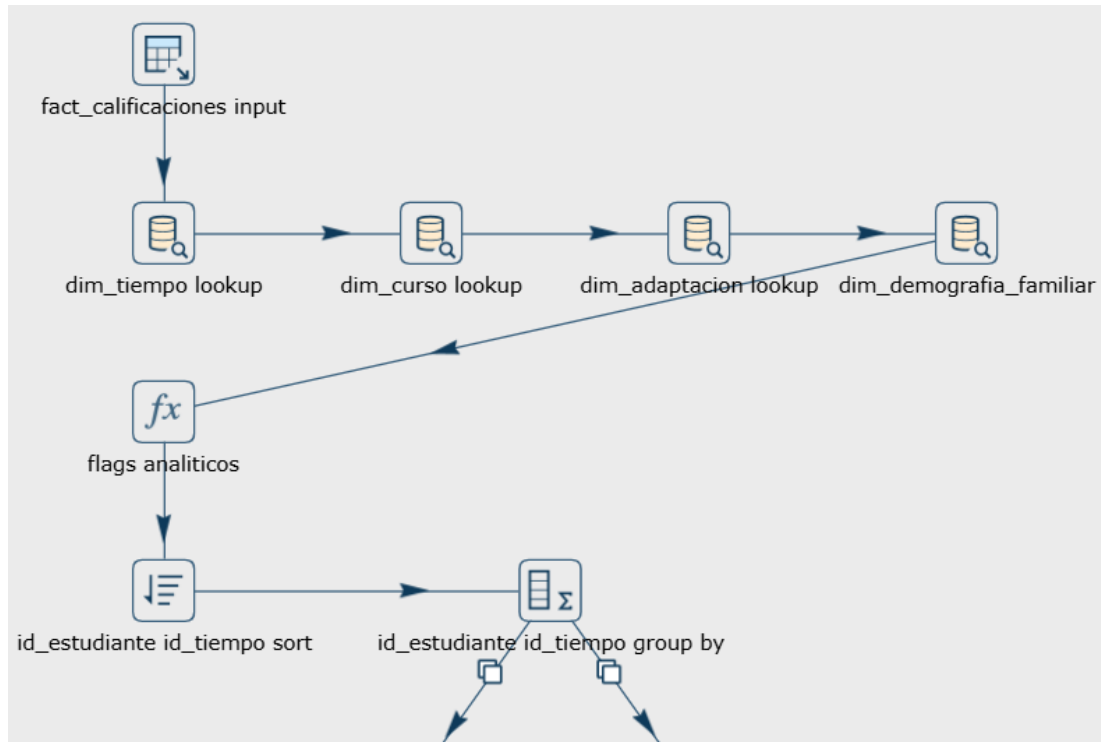


Figura 6.12: Flujo del pipeline *fact_rendimiento_anual_full.hpl* (Parte 1).

1. **dimensions.hwf**: Este flujo se encarga de la extracción inicial y la carga de dimensiones (ver Figura 6.17). Su ejecución comienza disparando el pipeline de extracción a *Staging*. Una vez los datos están en bruto en el esquema *raw*, el workflow lanza en **paralelo** la carga de todas las dimensiones (*dim_tiempo*, *dim_curso*, *dim_estudiante*, etc.) para maximizar el rendimiento. Es crítico que todas finalicen con éxito para garantizar la integridad referencial.
2. **facts.hwf**: Tras asegurar las dimensiones, *main.hwf* dispara este sub-workflow secuencial (ver Figura 6.18). Primero ejecuta el flujo atómico de calificaciones. Si finaliza con éxito, un nodo *Simple Evaluation* inspecciona el valor del parámetro $\{CURSO_ACTUAL\}$. Si la variable está vacía, el orquestador enruta la ejecución hacia el pipeline de rendimiento *Full* (carga masiva en memoria). Si contiene un curso específico, enruta dinámicamente hacia el pipeline *Incremental* (arquitectura híbrida base de datos-memoria), garantizando la máxima eficiencia operativa por evaluación.

6.3.1 Propagación de parámetros dinámicos

Una de las características más potentes de la orquestación implementada es la propagación jerárquica de variables. El proyecto evita el uso de valores "quemados" (*hardcoded*) en código,

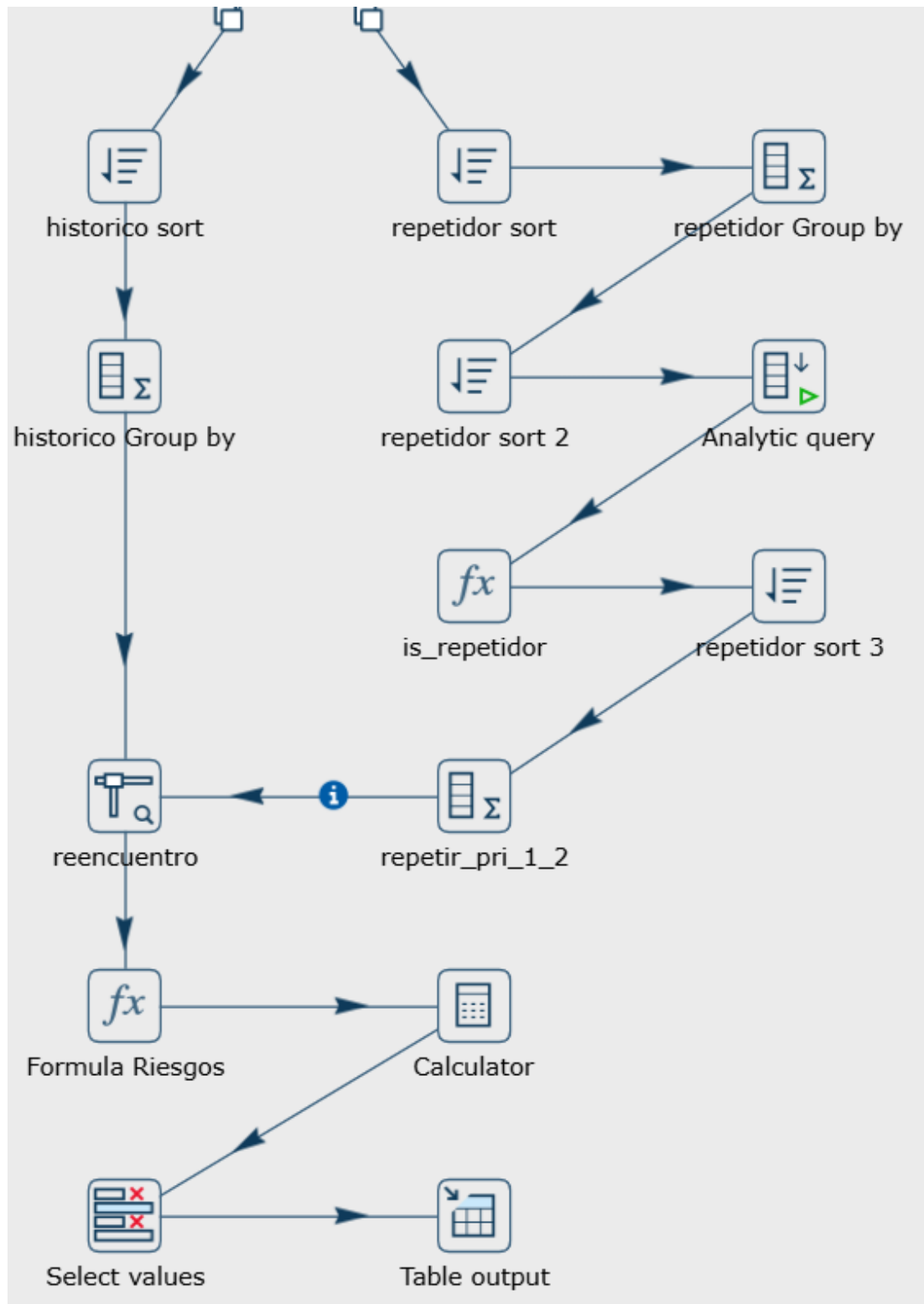


Figura 6.13: Flujo del pipeline *fact_rendimiento_anual_full.hpl* (Parte 2).

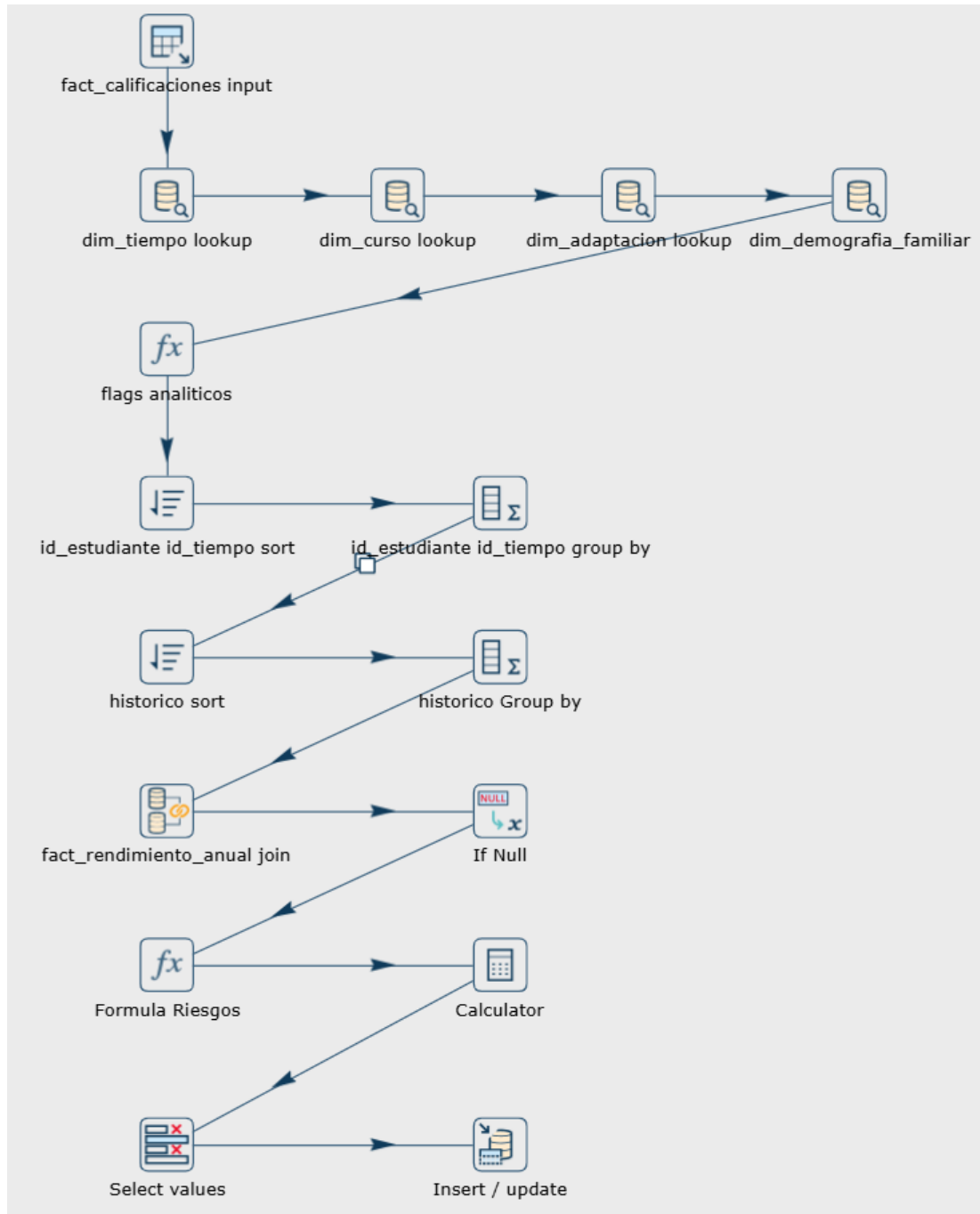
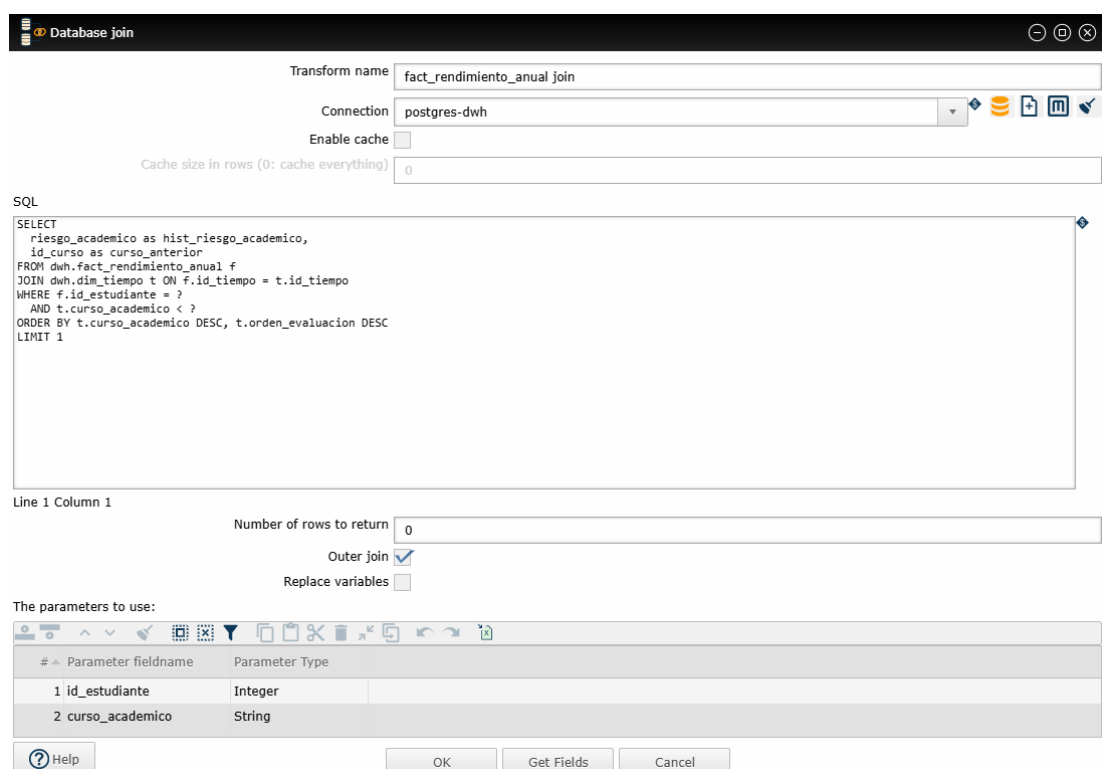


Figura 6.14: Flujo completo del pipeline `fact_rendimiento_anual_incremental.hpl`.



Transform name: fact_rendimiento_anual join

Connection: postgres-dwh

Enable cache: ☐

Cache size in rows (0: cache everything): 0

SQL

```
SELECT
  riesgo_academico as hist_riesgo_academico,
  id_curso as curso_anterior
FROM dwh.fact_rendimiento_anual f
JOIN dwh.dim_tiempo t ON f.id_tiempo = t.id_tiempo
WHERE f.id_estudiante = ?
AND t.curso_academico < ?
ORDER BY t.curso_academico DESC, t.orden_evaluacion DESC
LIMIT 1
```

Line 1 Column 1

Number of rows to return: 0

Outer join: ☒

Replace variables: ☐

The parameters to use:

#	Parameter fieldname	Parameter Type
1	id_estudiante	Integer
2	curso_academico	String

Buttons: ? Help, OK, Get Fields, Cancel

Figura 6.15: Nodo *Database Join* en *fact_rendimiento_anual_incremental.hpl*.

utilizando en su lugar el parámetro global *CURSO_ACTUAL* para gobernar el comportamiento de carga (Full o Incremental).

El ciclo de vida de este parámetro fluye en cascada o *top-down*:

1. **Inyección Externa:** El parámetro se define en el momento de la ejecución desde fuera del contenedor (por ejemplo, a través de variables de entorno en el *docker-compose* o argumentos del CLI de *Hop Run*).
2. **Recepción en el Workflow Principal:** El flujo *main.hwf* está configurado para declarar y capturar este valor en sus propiedades de ejecución, registrándolo en la memoria del orquestador.
3. **Transmisión a Sub-workflows:** A través de la configuración "*Pass parameter values to sub-workflow*", el *main* inyecta el curso actual hacia abajo, entregándolo como contexto válido a *dimensions.hwf* y *facts.hwf*.
4. **Consumo en los Pipelines:** Finalmente, los pipelines atómicos recogen la variable dinámica (invocada como $\${CURSO_ACTUAL}$) y la inyectan directamente en el código SQL de los nodos *Table Input* para filtrar fechas, o la evalúan lógicamente en pasos de enrutamiento.

Este diseño asegura que todo el ecosistema de decenas de pipelines adapte su comportamiento instantáneamente a un nuevo año académico alterando un único valor externo de ejecución.

6.3.2 Manejo de errores multinivel (*error handling*)

La robustez del sistema recae en un diseño de gestión de excepciones presente en todas las capas. Tanto el *main.hwf* como los sub-workflows (*dimensions.hwf* y *facts.hwf*) implementan bloques de captura de errores.

Si cualquier pipeline de transformación individual reporta un fallo (por ejemplo, una clave foránea huérfana), el sub-workflow correspondiente detiene su rama de ejecución y se desvía hacia una acción *Write to log*. Allí registra la incidencia ("Uno de los pipelines de dimensión ha fallado") y lanza un *Abort workflow*. Este fallo se propaga jerárquicamente hacia arriba hasta el *main.hwf*, que también registra su propio error crítico y aborta la orquestación global. Esta topología previene cualquier corrupción o carga parcial en el Data Warehouse.

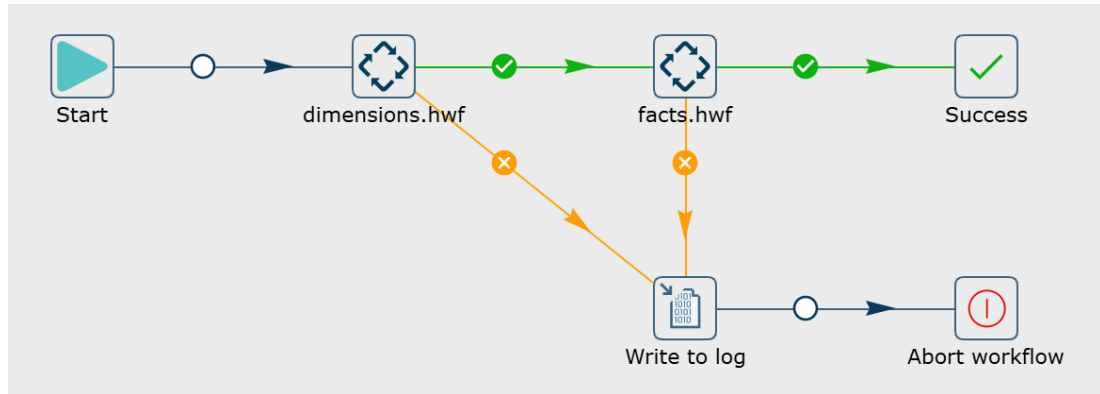


Figura 6.16: Workflow principal (*main.hwf*) y su delegación jerárquica en sub-flujos.

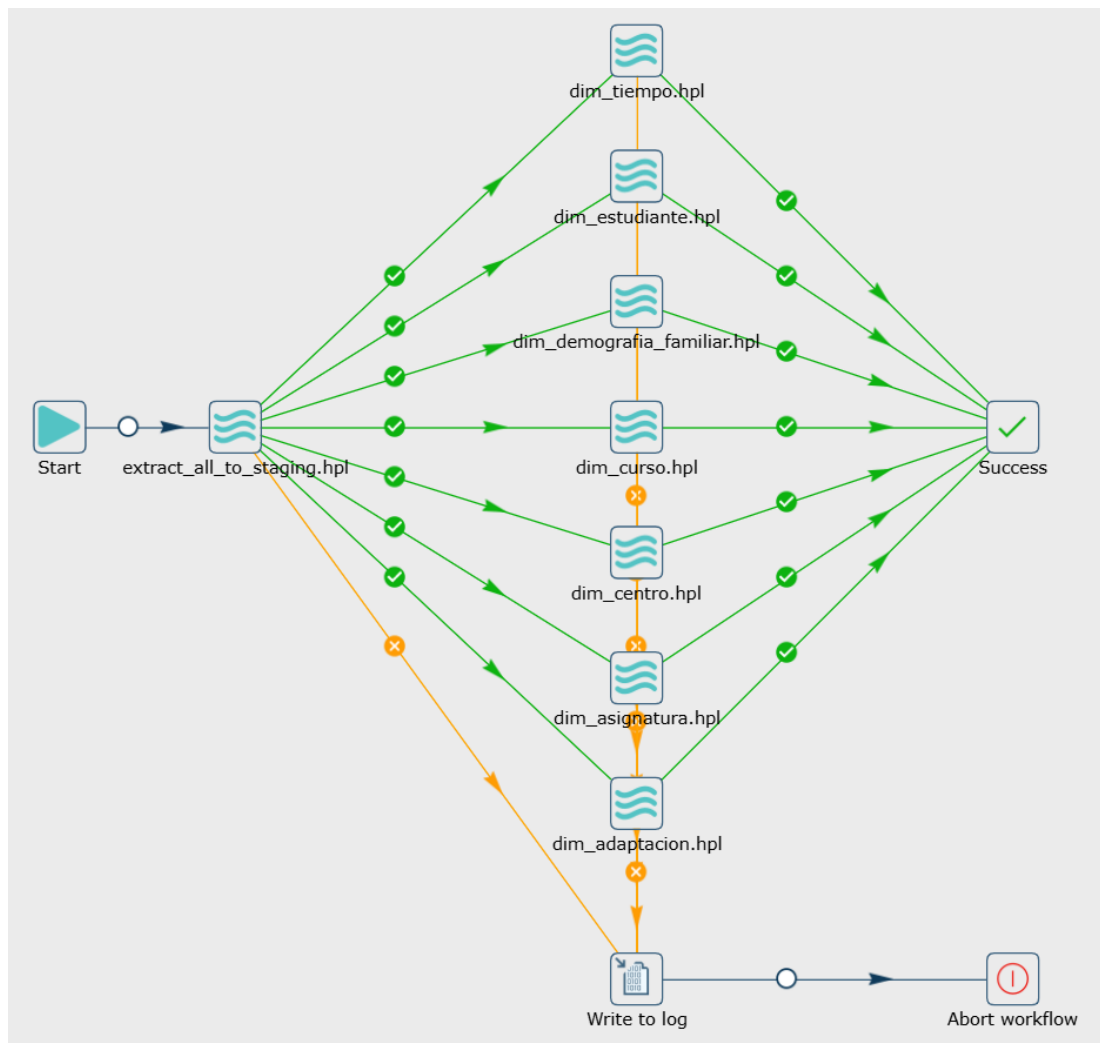


Figura 6.17: Sub-flujo de dimensiones (*dimensions.hwf*) ejecutado en paralelo.

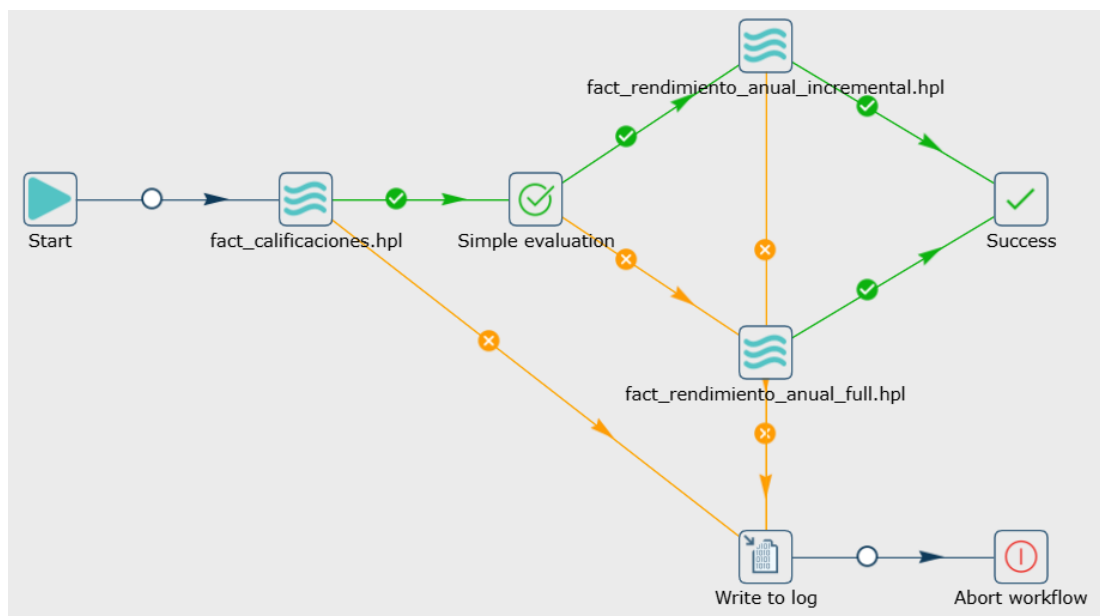


Figura 6.18: Sub-flujo de hechos (*facts.hwf*) ejecutado de forma secuencial.

Dashboard analítico

TRAS la consolidación de los datos en el Data Warehouse, la capa final del proyecto consiste en la creación de una aplicación web interactiva que permita a los gestores de la administración pública consumir esta información en tiempo real para tomar decisiones estratégicas.

Para cumplir con los requisitos de modularidad y escalabilidad (aislamiento de procesos), esta fase de presentación y acceso se ha dividido en tres contenedores independientes: el servidor de lógica de negocio (Backend), la interfaz visual (Frontend) y el servidor de enrutamiento web (Proxy Inverso).

7.1 Servidor backend RESTful (FastAPI)

El núcleo de la lógica de negocio se ha programado en Python utilizando el framework **FastAPI**. La elección de FastAPI frente a alternativas como Django o Flask se fundamenta en su alto rendimiento nativo (basado en *Starlette*), su soporte asíncrono y la generación automática de documentación interactiva mediante Swagger UI, lo que acelera enormemente la fase de integración frontend-backend.

7.1.1 Mapeo objeto-relacional (SQLAlchemy)

Dado que el Data Warehouse (DWH) es de solo lectura para el backend —su escritura es competencia exclusiva de la ETL de Apache Hop—, se ha creado una clase declarativa independiente (*DWHBase*) explícitamente configurada en el esquema *dwh*. Utilizando la librería **SQLAlchemy**, se han mapeado las tablas dimensionales y de hechos como objetos de Python (*Object-Relational Mapping*).

Esta abstracción permite construir consultas analíticas dinámicas (por ejemplo, cruces entre calificaciones históricas, demografía familiar y perfiles académicos del alumnado) de forma

programática, garantizando la seguridad frente a inyecciones SQL sin acoplar fuertemente el código a la estructura del motor de base de datos.

7.1.2 Estructura de endpoints y seguridad

La arquitectura de la API (*Routing*) se ha dividido lógicamente por dominios de negocio. El enrutador principal (*dashboard.py*) expone servicios protegidos que retornan información altamente agregada (KPIs), agrupaciones para series temporales (Charts) y paginación masiva de estudiantes en formato estandarizado JSON.

Para asegurar la máxima privacidad de los datos académicos y cumplir con las normativas de protección de datos infantiles, todos los *endpoints* requieren autenticación obligatoria. Se ha implementado un sistema robusto de tokens basado en el estándar **JWT** (JSON Web Tokens), inyectado en el ciclo de vida de las peticiones mediante el mecanismo de dependencias nativo de FastAPI (*Depends(get_current_user)*).

7.2 Interfaz visual e interacción (React y Vite)

La capa visual de cara al usuario ha sido desarrollada como una aplicación de página única (*Single Page Application* o SPA) utilizando la librería **React**. La construcción y empaquetado del proyecto se ha orquestado bajo el moderno motor **Vite**, lo que optimiza radicalmente los tiempos de compilación y carga frente a alternativas monolíticas tradicionales como Webpack.

7.2.1 Sistema de diseño: *Glassmorphism* y temas dinámicos

Para ofrecer una experiencia de usuario (UX) inmersiva y de calidad *Enterprise*, se ha prescindido de librerías de estilos genéricas (como Bootstrap), implementando en su lugar un sistema de diseño propietario desarrollado desde cero con CSS puro. La estética visual se inspira en la técnica **Glassmorphism**, popularizada en los ecosistemas operativos modernos como macOS y Windows 11.

Técnicamente, este efecto se logra aplicando propiedades avanzadas de desenfoque de fondo (*backdrop-filter: blur()*) combinadas con tarjetas semitransparentes. Esta composición genera una ilusión óptica de jerarquía y profundidad sobre un tapiz de colores vivos subyacentes.

Asimismo, el sistema implementa soporte nativo para el **Modo Oscuro** (*Dark Mode*). Mediante el uso de variables CSS dinámicas inyectadas como atributos de datos HTML (`['data-theme="dark"]'`), el sistema recalcula toda la paleta de colores y sombreados instantáneamente, garantizando el confort visual durante su uso prolongado en los despachos de la administración pública.

7.2.2 Visualización de datos analíticos

La complejidad real del frontend radica en la asimilación del volumen de datos del backend. Para la renderización de métricas y proyecciones, se ha integrado la potente librería de gráficos declarativos **Recharts**. Los datos masivos procedentes de la API —como la distribución del riesgo por ciclos formativos o la pirámide de impacto del nivel educativo parental— se inyectan en componentes dinámicos encapsulados bajo el componente propietario *GlassCard*, asegurando una presentación limpia y libre de ruidos visuales.

7.3 Despliegue y enrutamiento web (Nginx)

El punto de entrada único de toda la arquitectura técnica frente al exterior se gestiona mediante un contenedor con **Nginx**, que actúa simultáneamente como servidor web de alto rendimiento y como *Proxy Inverso*.

7.3.1 Arquitectura de interceptación de tráfico

El contenedor Nginx es la única pieza del ecosistema que expone sus puertos directamente a la red de la administración (típicamente los puertos HTTP 80 y HTTPS 443). Cuando un usuario accede al sistema desde su navegador, el comportamiento de enrutamiento se bifurca según la ruta (*path*) solicitada:

- **Tráfico Estático (Ruta /):** Si la ruta solicitada corresponde a la carga inicial de la aplicación, Nginx intercepta la petición y sirve directamente de su memoria los ficheros compilados de producción (HTML, CSS y JS minimizados) generados por el contenedor de React, maximizando la velocidad de acceso.
- **Tráfico API (Ruta /api/*):** Si la aplicación React realiza una solicitud asíncrona de datos, Nginx actúa como un túnel transparente de proxy inverso. Captura la petición web, la reescribe y la deriva al contenedor interno de FastAPI que yace escondido en la red de Docker. Posteriormente recoge el JSON de respuesta y se lo entrega al cliente original.

Esta arquitectura en capas no solo representa una práctica estandarizada de máxima eficiencia en la industria DevOps actual, sino que aísla el delicado servidor de lógica (FastAPI) y la base de datos de la red pública, blindando la aplicación contra ataques directos e intentos de explotación.

Conclusións

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras,...

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografía

- [1] Organización para la Cooperación y el Desarrollo Económicos (OCDE), “Education at a glance: Oecd indicators,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://www.oecd.org/education/education-at-a-glance/>
- [2] Instituto Nacional de Estadística (INE), “Indicadores de pobreza y condiciones de vida,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://www.ine.es/daco/daco42/sociales/pobreza.pdf>
- [3] Ayuda en Acción, “Pobreza absoluta y pobreza relativa: diferencias,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://ayudaenaccion.org/blog/solidaridad/pobreza-absoluta-pobreza-relativa/>
- [4] BBVA Sostenibilidad, “¿qué es la pobreza multidimensional y cómo se mide?” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://www.bbva.com/es/sostenibilidad/que-es-la-pobreza-multidimensional-y-como-se-mide/>
- [5] Observatorio de la Pobreza, “Observatorio de la pobreza y la desigualdad,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://www.observatoriopobreza.org/>
- [6] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. John Wiley & Sons, 2013.
- [7] The Apache Software Foundation, “Apache hop (hop orchestration platform) documentation,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://hop.apache.org/manual/latest/>
- [8] Docker Inc., “Docker documentation,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://docs.docker.com/>

- [9] The PostgreSQL Global Development Group, “Postgresql 16.0 documentation,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://www.postgresql.org/docs/16/index.html>
- [10] S. Ramírez, “Fastapi documentation,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://fastapi.tiangolo.com/>
- [11] Meta Platforms, Inc., “React: The library for web and native user interfaces,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://react.dev/>
- [12] OECD, “Pisa 2018 technical report,” 2020, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://www.oecd.org/pisa/data/pisa2018technicalreport/>
- [13] Instituto Nacional de Estadística (INE), “Encuesta de condiciones de vida (ecv),” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: https://www.ine.es/dyngs/INEbase/es/operacion.htm?c=Estadistica_C&cid=1254736176807&menu=resultados&idp=1254735976608
- [14] Red Europea de Lucha contra la Pobreza y la Exclusión Social en el Estado Español (EAPN-ES), “Guía para comprender la pobreza,” 2023, consultado el 24 de junio de 2026. [En línea]. Disponible en: <https://eapn-clm.org/guia-para-comprender-la-pobreza/>